



Agentic Systems: Part II

Vector DBs & Retrieval Augmented Generation

Presented by

Anirudh Krishnan, Evan Zimmerman, Naveen Unnikrishnan, Pranav Mallela



METIS: Fast Quality-Aware RAG Systems with Configuration Adaptation

Siddhant Ray, Rui Pan, Zhuohan Gu et al.

SOSP 2025

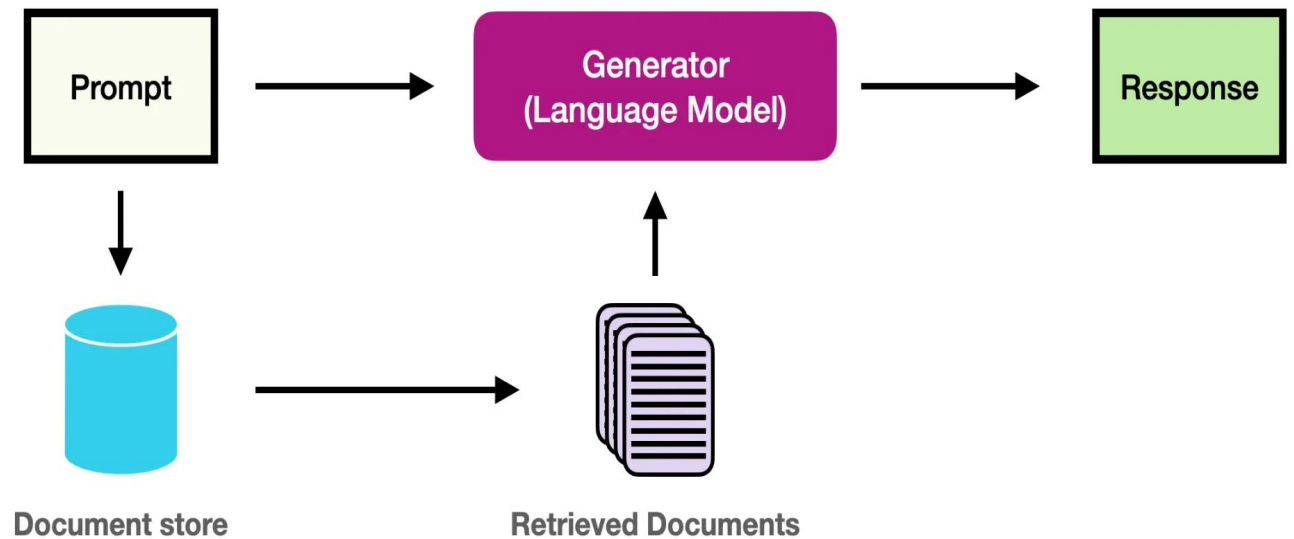
What is RAG ?

RAG = Retrieval Augmented Generation:

Augmenting prompts with external knowledge to generate high quality responses.

Use Case: Q & A Workflows that require data unavailable to the LLM

Retrieval Augmented Generation



What is RAG ?

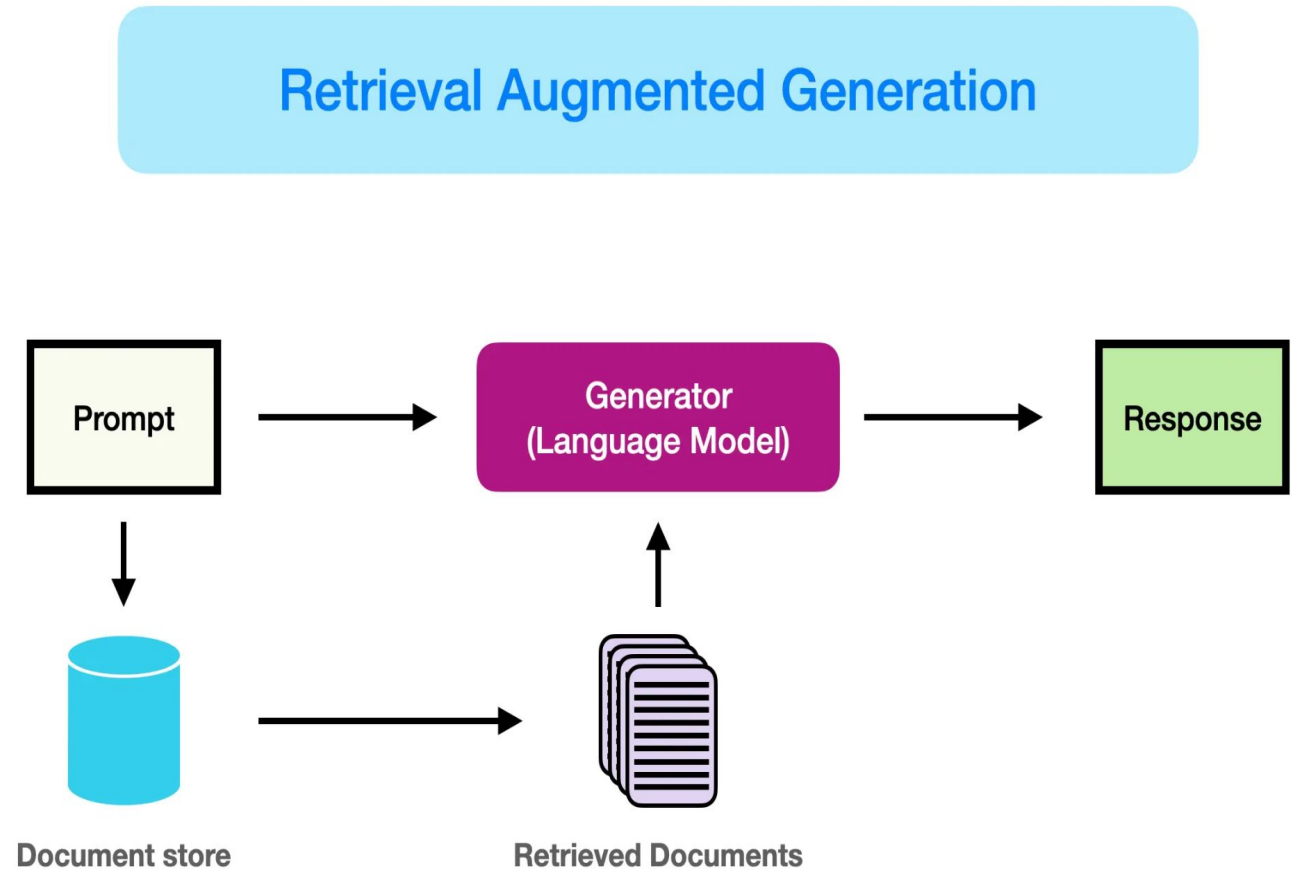
RAG = Retrieval Augmented Generation:

Augmenting prompts with external knowledge to generate high quality responses.

Use Case: Q & A Workflows that require data unavailable to the LLM

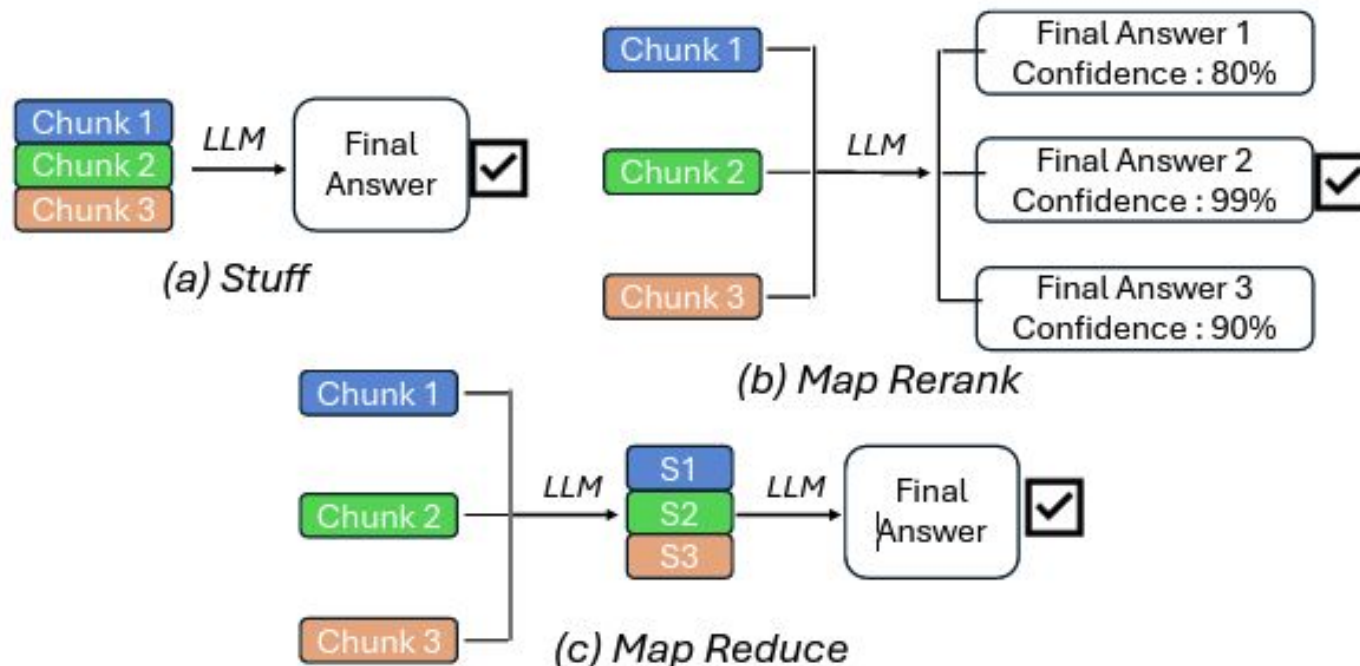
How do we find the right external knowledge?

- *Set up:* store document chunks as embeddings in a vector DB
- *Retrieve:* Find the chunks most similar to our prompt embedding by measuring vector similarity
- *Synthesize:* Combine the chunks to form a new query that gets sent to the LLM



Synthesis Methods

- (a) **Stuff:** Concatenate all chunks in a single prompt
- (b) **Map Rerank:** Independently generate one answer/chunk and then pick the answer with the most confidence
- (c) **Map Reduce:** Summarize each chunk before doing Stuff



The Goal of RAG Systems

What we want: (1) High quality responses and (2) Low response delay

Problem: These two goals are often in conflict with each other because higher quality responses often require more computation but lead to longer delays.

Existing Approaches: Prior works try to optimize for (1) OR (2) at the expense of the other (i.e. AdaptiveRag [1]).

METIS' Approach: Jointly optimize for the quality-delay tradeoff

The Goal of RAG Systems

What we want: (1) High quality responses and (2) Low response delay

Problem: These two goals are often in conflict with each other because higher quality responses often require more computation but lead to longer delays.

The Goal of RAG Systems

What we want: (1) High quality responses and (2) Low response delay

Problem: These two goals are often in conflict with each other because higher quality responses often require more computation but lead to longer delays.

Prior works try to optimize for (1) OR (2) at the expense of the other (i.e. AdaptiveRag [1]).

The Goal of RAG Systems

What we want: (1) High quality responses and **(2) Low response delay**

Problem: These two goals are often in conflict with each other because higher quality responses often require more computation but lead to longer delays.

Prior works try to optimize for (1) OR **(2)** at the expense of the other (i.e. AdaptiveRag [1]).

The Goal of RAG Systems

What we want: (1) High quality responses and (2) Low response delay

Problem: These two goals are often in conflict with each other because higher quality responses often require more computation but lead to longer delays.

METIS' Approach

Jointly optimize for the quality-delay tradeoff

Configuration Questions

Changing each knob comes with trade-offs

How many chunks to retrieve?

Too few chunks: faster synthesis and generation, but risks less accuracy

Too many chunks: slower synthesis and generation, but potentially more accuracy

Configuration Questions

Changing each knob comes with trade-offs

How many chunks to retrieve?	How to synthesize chunks?
Too few chunks: faster synthesis and generation, but risks less accuracy	Map Rerank: fast because of less computation, but risks low accuracy
Too many chunks: slower synthesis and generation, but potentially more accuracy	Stuff: slower than map rerank, but offers potentially better accuracy up until a point (<i>lost-in-the-middle-problem</i>). Map Reduce: slower because of summarization, but offers better context quality than stuff .

Configuration Questions

Changing each knob comes with trade-offs

How many chunks to retrieve?	How to synthesize chunks?	How much to summarize?
Too few chunks: faster synthesis and generation, but risks less accuracy Too many chunks: slower synthesis and generation, but potentially more accuracy	Map Rerank: fast because of less computation, but risks low accuracy Stuff: slower than map rerank, but offers potentially better accuracy up until a point (<i>lost-in-the-middle-problem</i>). Map Reduce: slower because of summarization, but offers better context quality than stuff.	Short summaries: faster to generate, risk losing more information Long summaries: slower to generate, but retain more information

Sample Queries for Comparing Configurations

Q1 (Simple): *“In what county was William W. Blair’s born?”*

Q2 (Medium): *“Are Alison Skipper, Diane Gilliam Fisher, and Rachel McAdams from the same country?”*

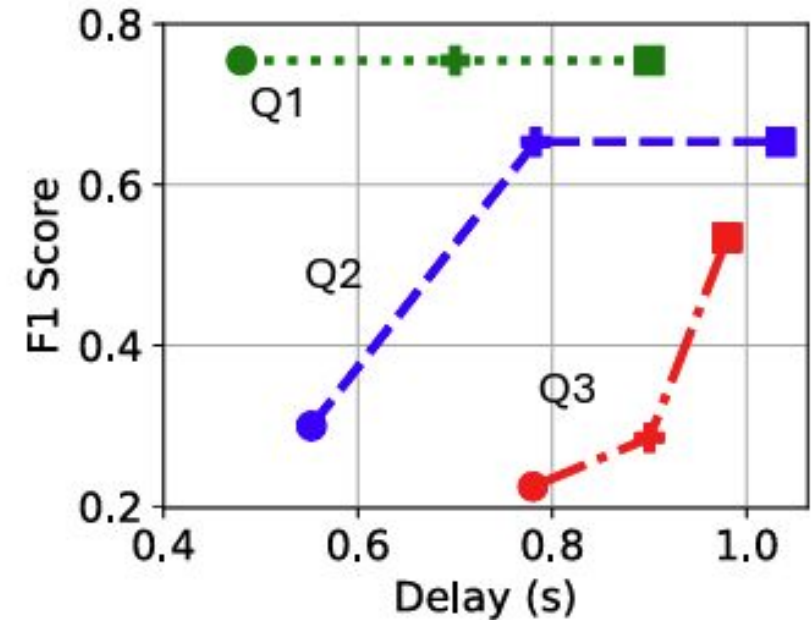
Q3 (Complex): *“When and why did the Voyager 1, the spacecraft that detected storms on Neptune, leave our solar system?”*

Impact of Synthesis Method

It depends on the query.

- **Simple queries:** don't benefit from *stuff* or *map reduce* because they don't require many chunks
- **Complicated queries:** benefit from *stuff* and *map reduce* (sort of)

Q1 (Simple) Q2 (Medium) Q3 (Complex)

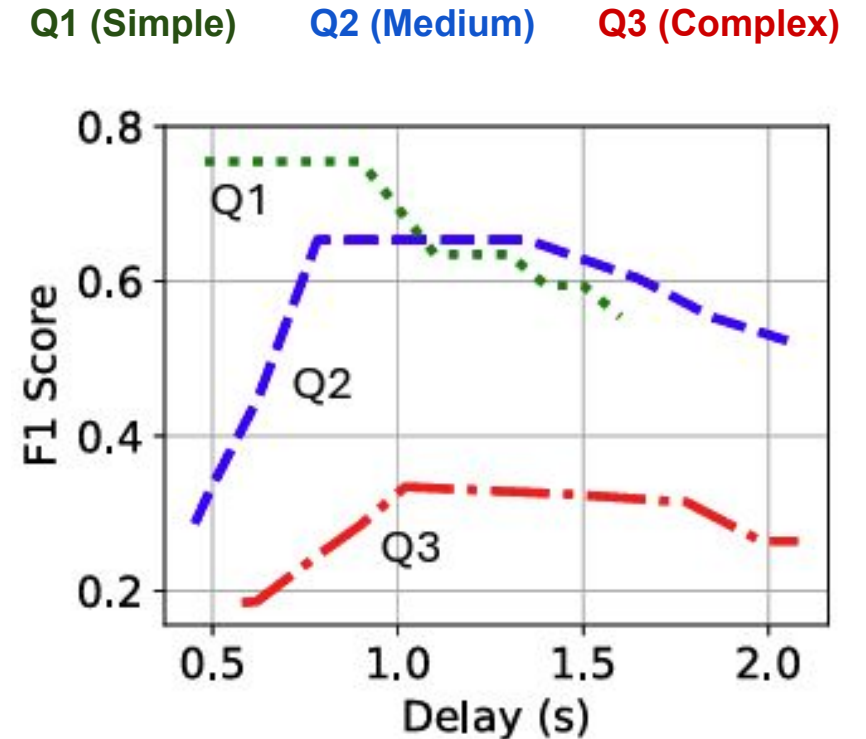


(a) Change: synthesis method from map_rerank (*circle*) , stuff (*plus*) and map_reduce (*square*)

Impact of # of Chunks

It depends on the query.

- **Simple queries:** increasing the # of chunks doesn't help
- **Complicated queries:** initially benefit from more chunks → diminishing returns → decline in accuracy



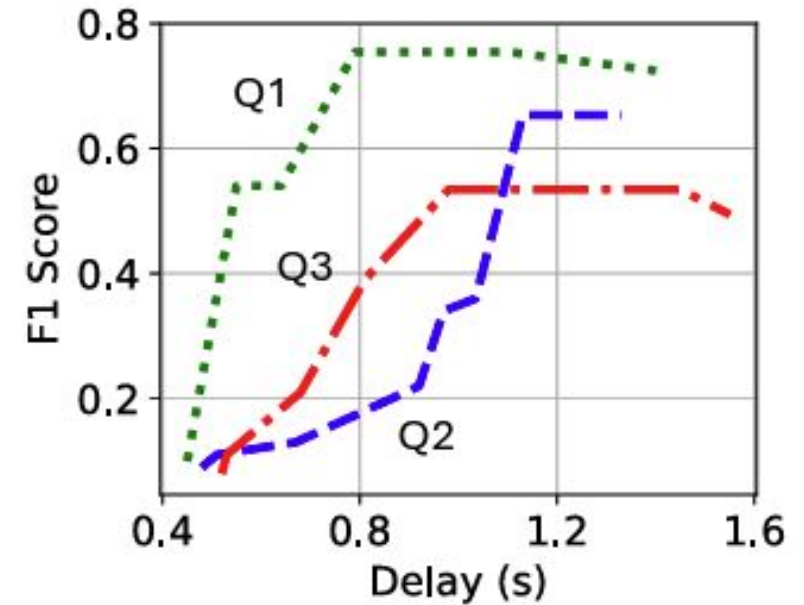
(b) Change: Increase # of chunks from 1 to 35 with stuff (left to right)

Impact of Summarization Length

It depends on the query.

- **Simple queries:** shorter summarizations are sufficient
- **Complicated queries:** longer summarizations initially improve accuracy, but the returns plateau

Q1 (Simple) Q2 (Medium) Q3 (Complex)



(c) Change: Increase intermediate length from 1 to 100 with map_reduce (left to right)

How to Configure Knobs ?

Per-Query RAG configurations offer the best results.

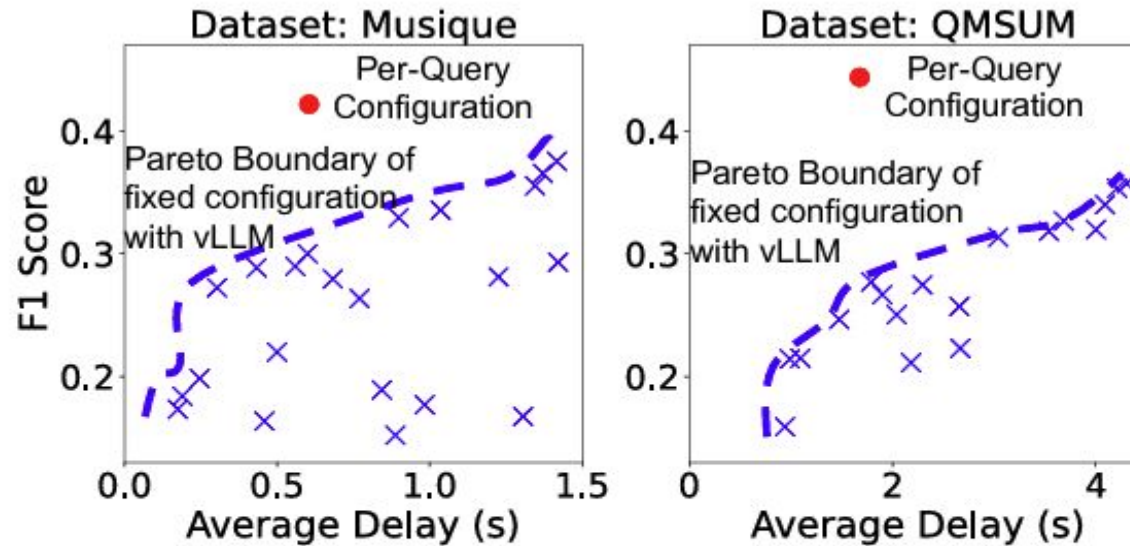


Figure 5. Per-query configuration can achieve significantly better quality-delay tradeoffs across queries compared to every fixed configuration choice.

Challenges to Per-Query Configurations

1. More knobs = Larger configuration space (exponential growth)

With Map Reduce for example, with n chunks and m summarization lengths we get $n * m$ possible configurations.

2. Profiling configurations is expensive

It requires many inference calls to the LLM, consuming valuable resources.

Challenges to Per-Query Configurations

1. More knobs = Larger configuration space (exponential growth)

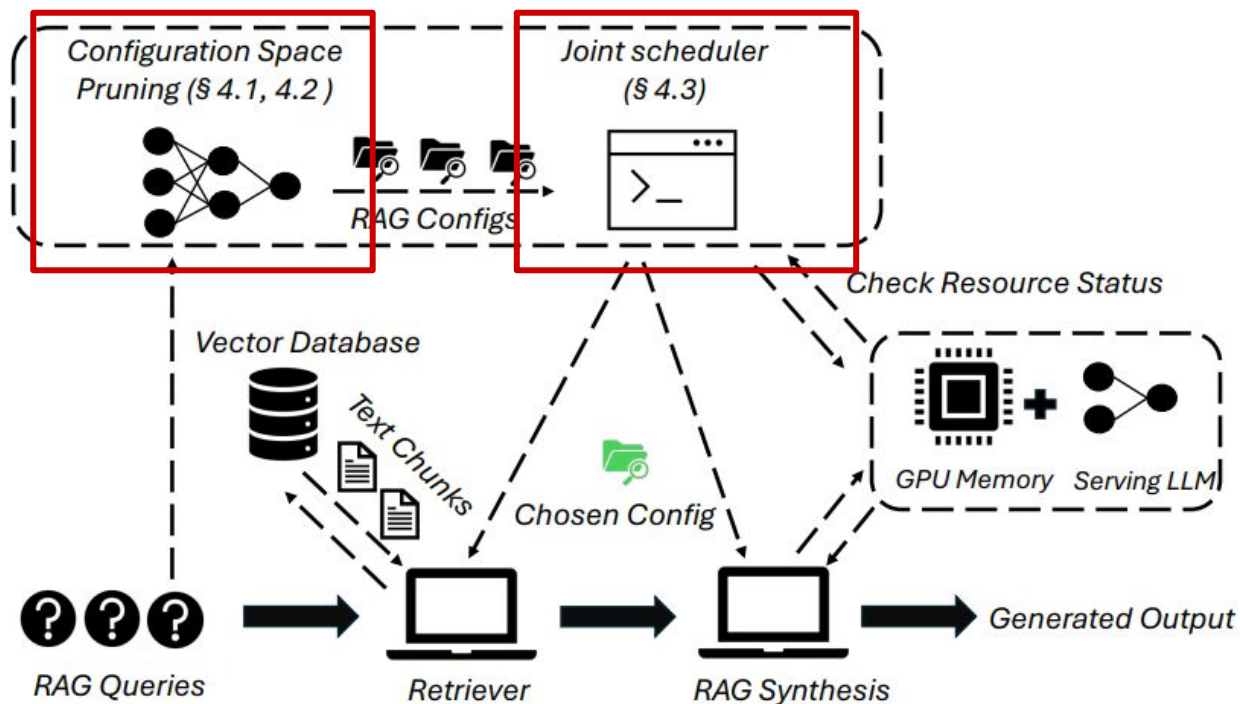
With Map Reduce for example, with n chunks and m summarization lengths we get $n * m$ possible configurations.

2. Profiling configurations is expensive

It requires many inference calls to the LLM, consuming valuable resources.

Takeaway: Need a system that can **quickly** and **cheaply** make decisions about per-query configurations over a large configuration space.

Approach



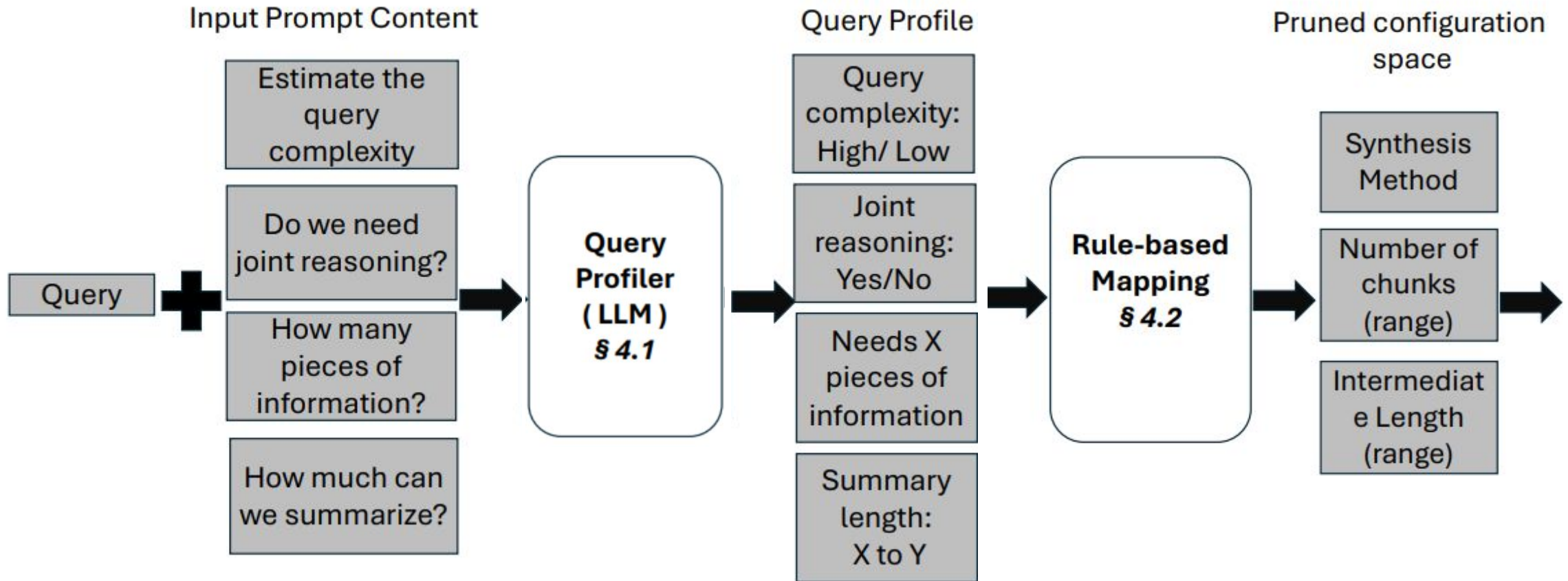
1. Pruning the configuration space!

Profile the query to understand which RAG configuration is best

2. Jointly optimize configuration and scheduler!

Optimize for both quality and latency by choosing the best configuration that enables the query to fit in memory

Pruning the Config Space



Query Profiler Cost

- The profiler is an expressive model which works with minimal information
- A much smaller model works for inference; it relies on retrieved context rather than model weights
- About 1/10 the delay of the RAG query

Rule-Based Mapping

Algorithm 1: Rule based mapping algorithm

Input: *Query complexity, Joint reasoning required*

Input: *Pieces of information , Summarization length range*

Result: *synthesis_method, num_chunks, intermediate_length*

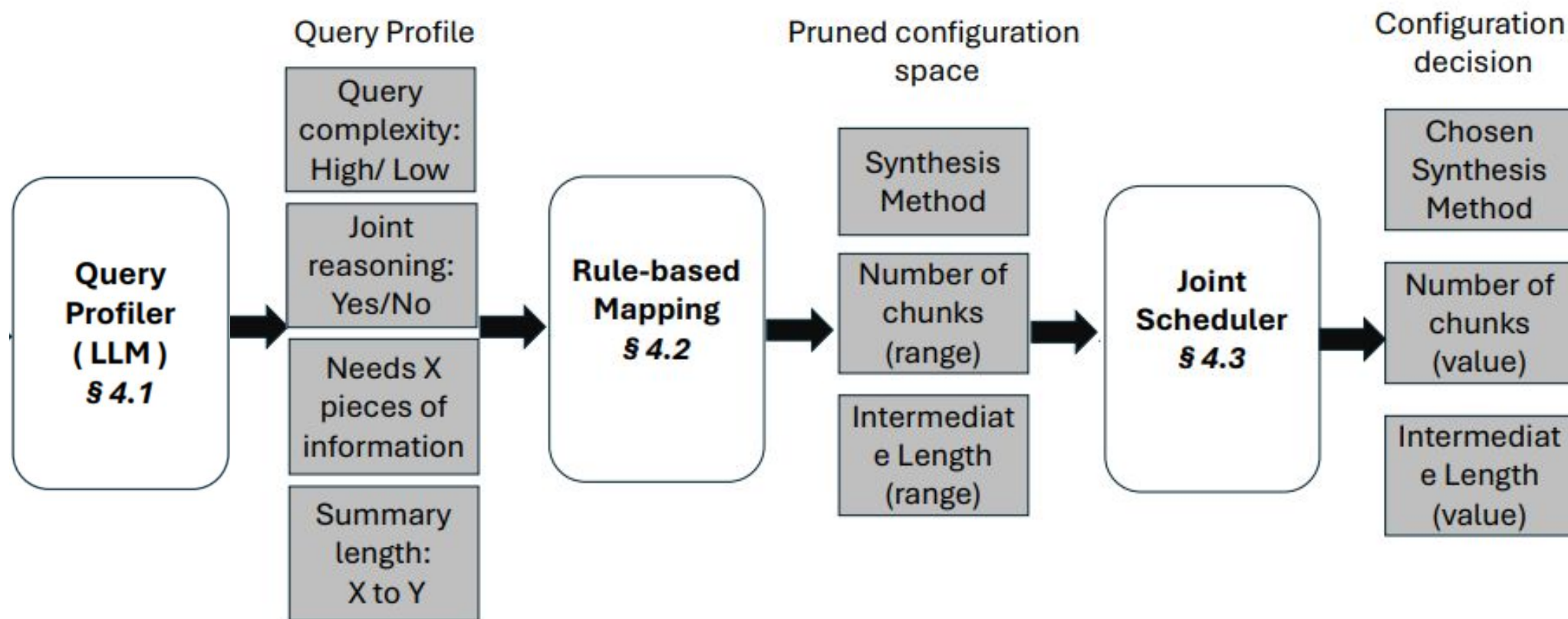
```
1 if Joint reasoning required == "no" then
2   | synthesis_method = map_rerank
3 else
4   | if Query complexity == "low" then
5     | synthesis_method = stuff
6   | else
7     | synthesis_method = stuff, map_reduce
8 num_chunks = [Pieces of information , 3× Pieces of information]
9 intermediate_length_range = Summarization length range
```

synthesis_method is being set here:

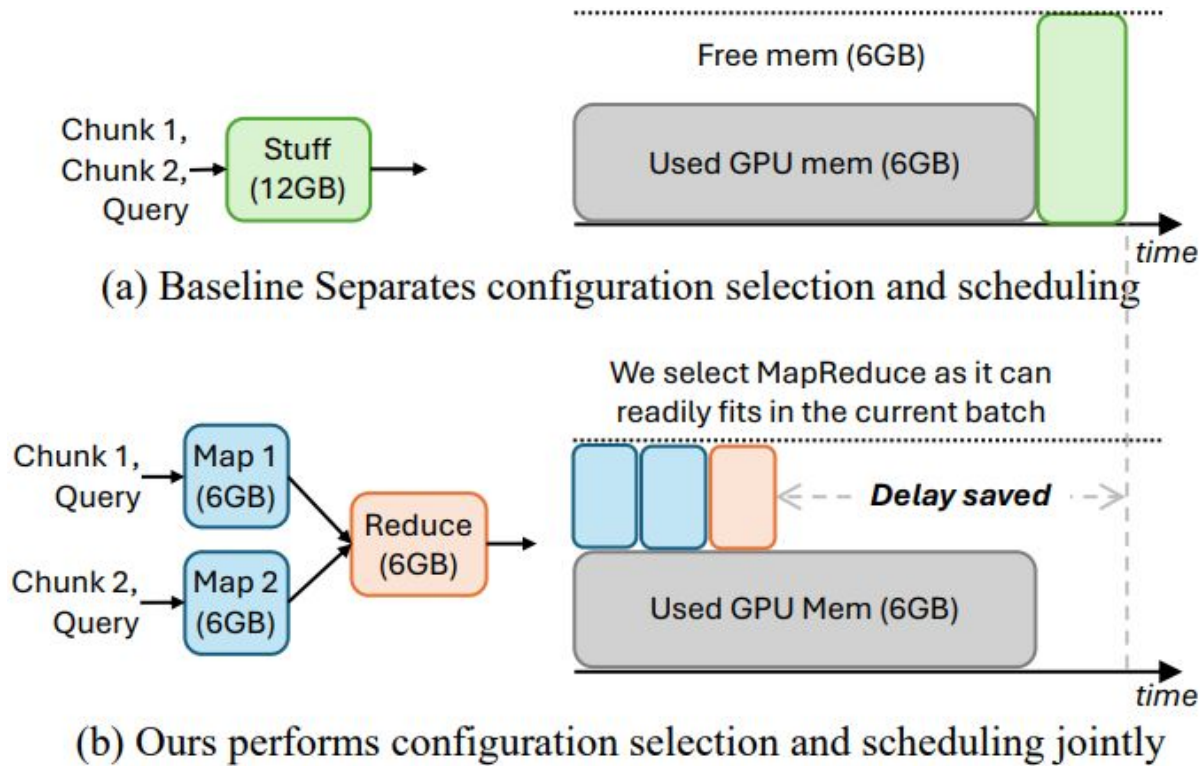
1. If joint reasoning is not required, use map_rerank.
2. If joint reasoning is required, and it's a simple query, use stuff.
3. If joint reasoning is required, and it's a complex query, use stuff or map_reduce.

num_chunks is set to a range: allows retrieval logic to find the right chunks and helps in scheduling.

Joint Scheduling



Joint Scheduling



Q. Why do we have to do this though?

Baseline that separates scheduling and configuration selection

- Chooses `stuff` as it has the lower compute requirement
- If the memory availability is low, it ends up being slower due to queuing.

METIS can choose to use `stuff` or `map_reduce`

- As it considers memory availability, it proceeds with `map_reduce` in the above scenario and completes the query with lower latency.

Joint Scheduling

Best Fit Algorithm:

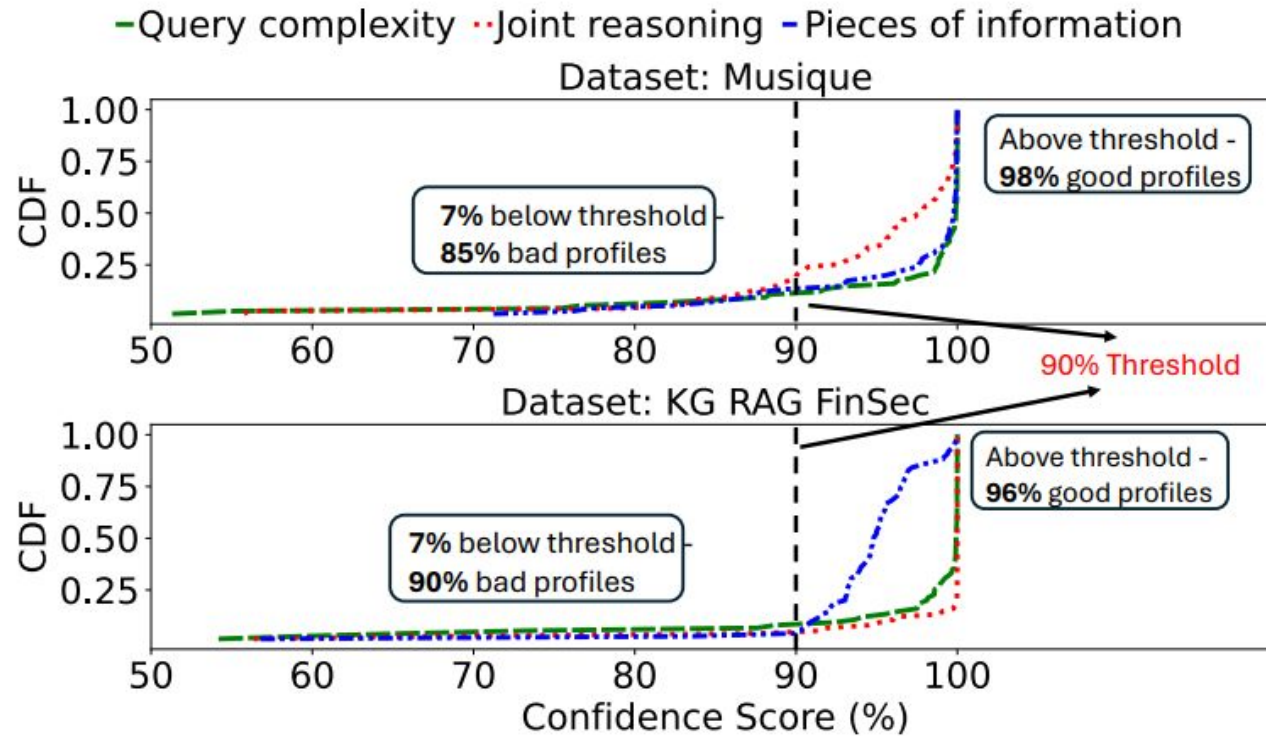
1. Compute GPU requirement of each query from input token length, parameters of inference model, quantization.
2. Keep a small 2% buffer size on top to deal with OOM crashes.
3. Pick best configuration: among the queries that fit in memory, the one with the highest memory requirement

If 5 or 6 chunks can fit in memory, allow 6 chunks!

If no configuration fits:

1. Fall back to cheaper option outside pruned range!

Refinements



1. When is the profiler output reliable?

When the inference model's answer is confident enough.

2. How to improve the profiler over time?

Ask the profiler what configuration it should choose based on the query and the answer synthesized using the most expensive configuration.

Evaluation Setup

1. Datasets:

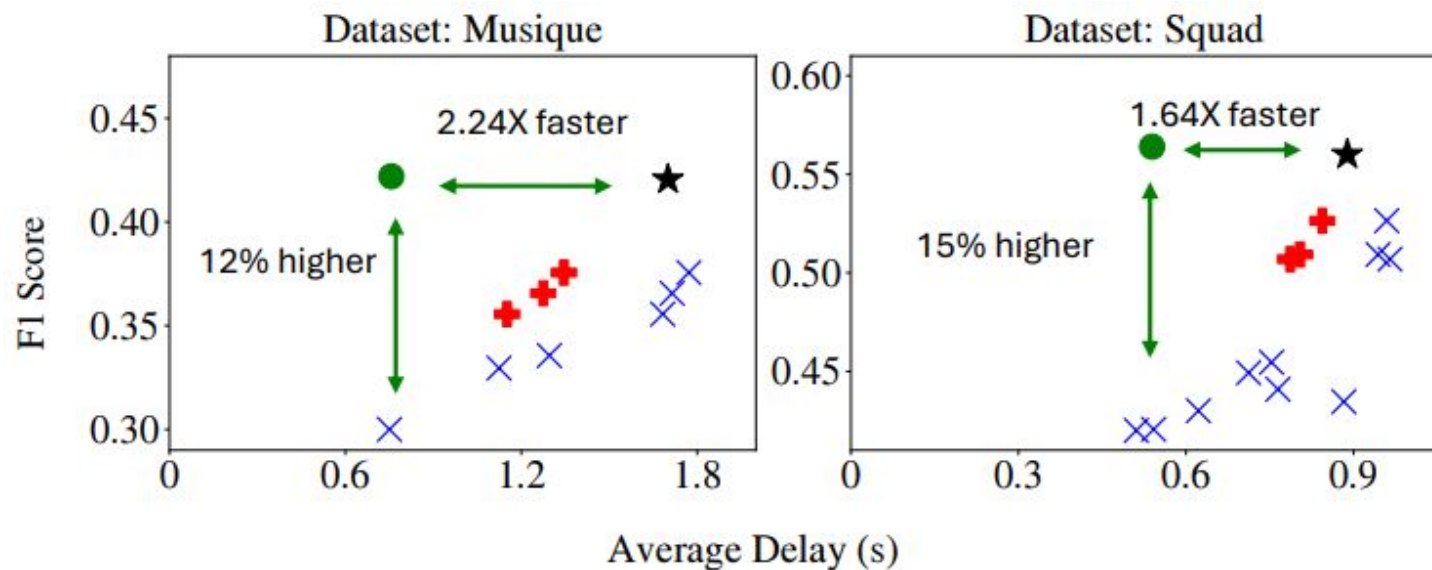
- a. **Squad:** Single-hop reading comprehension dataset
- b. **Musique:** Multi-hop QA dataset with reasoning-based questions
- c. **FinSec:** Financial domain questions from Fortune 500 companies
- d. **QMSUM:** Multi-domain meeting summarization benchmark.

2. Models: Commonly used serving models for RAG QA Evaluation: Smaller, instruction-tuned models such as Mistral-7B-v3 and Llama3.1-70B

3. Metrics:

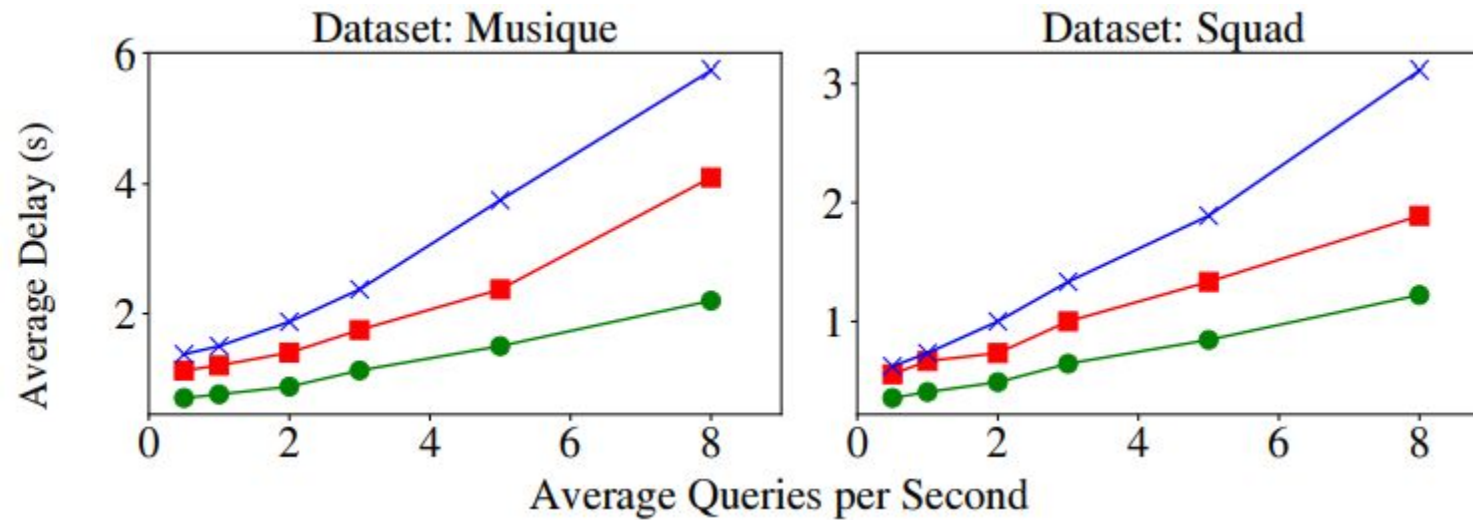
- a. **Quality:** F1 score
- b. **System:** Delay, Dollar Cost

Results: Average Latency



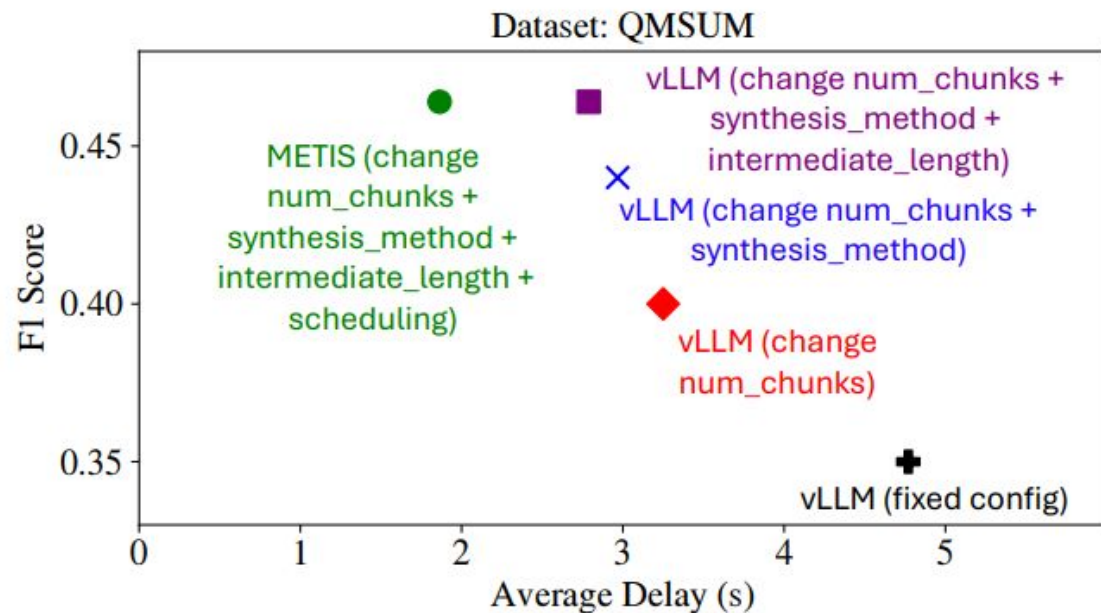
● METIS (w/ adapted RAG config and batching) + Parrot* (w/ fixed RAG config) ★ AdaptiveRAG* (selected config w/ Parrot) × vLLM (w/ fixed RAG config)

Results: Average Throughput

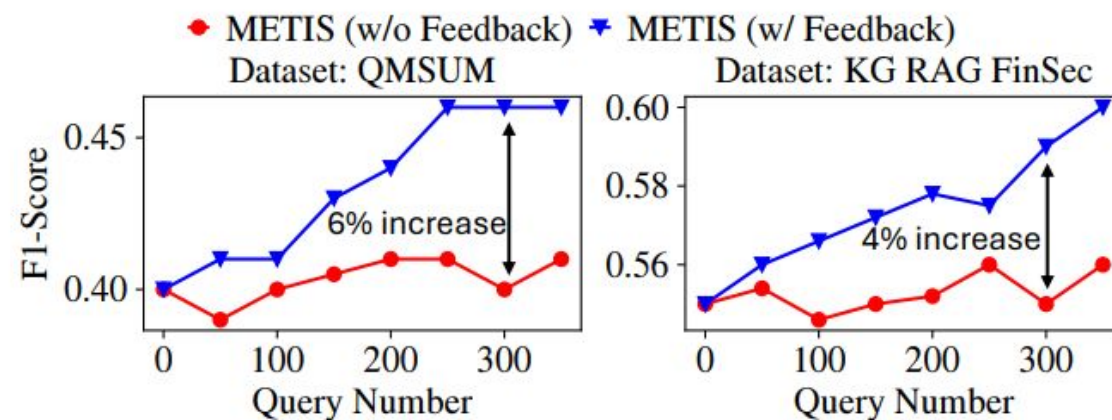


● METIS (w/ adapted RAG config and batching) + Parrot* (w/ fixed RAG config) ★ AdaptiveRAG* (selected config w/ Parrot) × vLLM (w/ fixed RAG config)

Results: Interesting Ablations



By tuning more knobs, we can see better performance.



Feedback prompting actually helps!

Limitations

- Rule based mapping is difficult to extend. *Do you just keep adding rules as you add more knobs?*
- Experiments are only ran against **one other** adaptive query configuration system (AdaptiveRAG)
- Results focus on RAG for Q & A, but nowadays RAG is often used for other tasks (i.e. Agentic RAG, multimodal RAG)

Future Work

1. Integrating other RAG Configurations:

- External Re-ranking
- Query rewriting
- Web Search

2. Integrating with LLM Routing

Another example of LLM outputs and mapping functions to guide real system decisions and optimizations.

- LLM Routing: use an LLM to choose the best family of models to serve a request
- How can RAG and routing be combined in the same system?

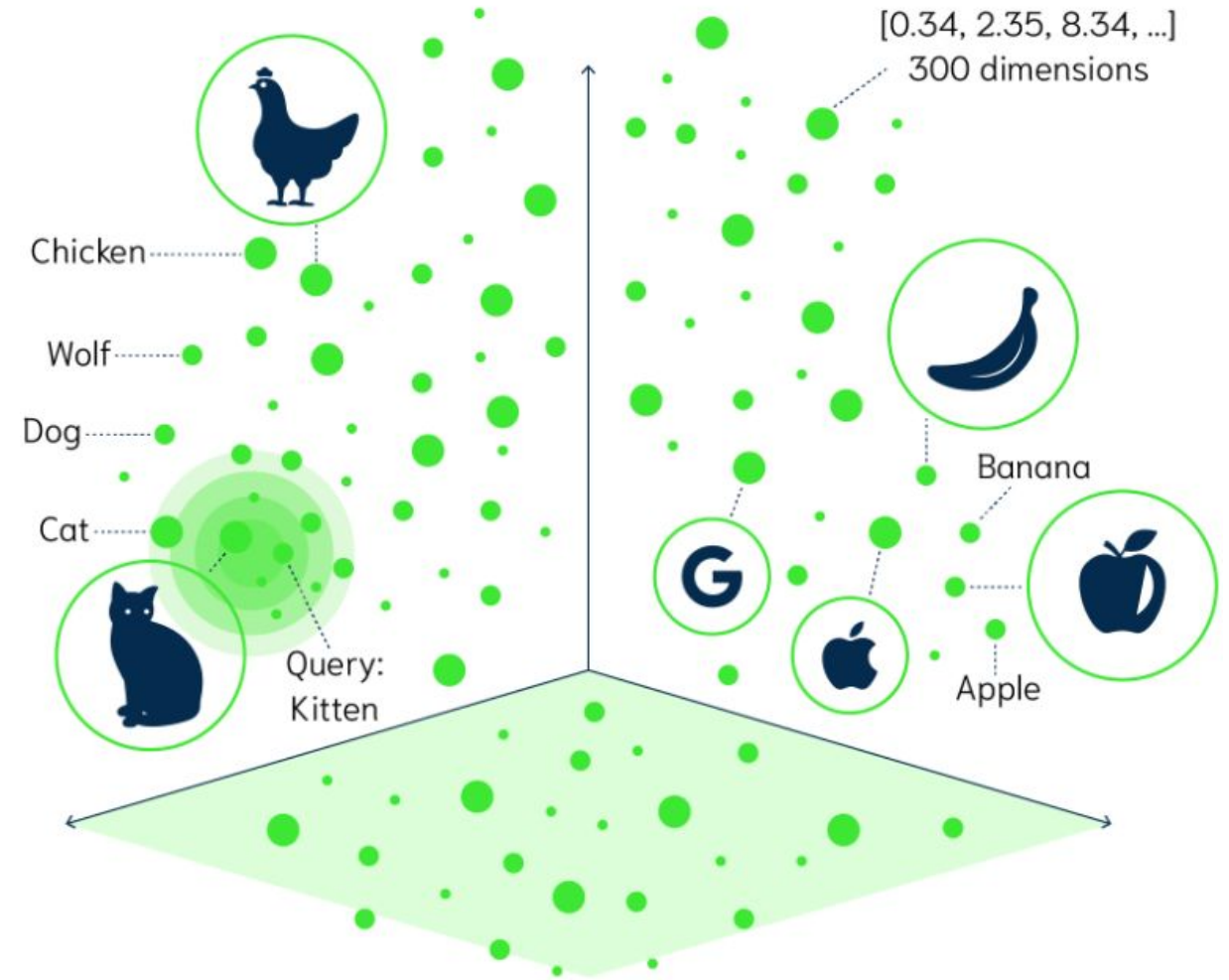
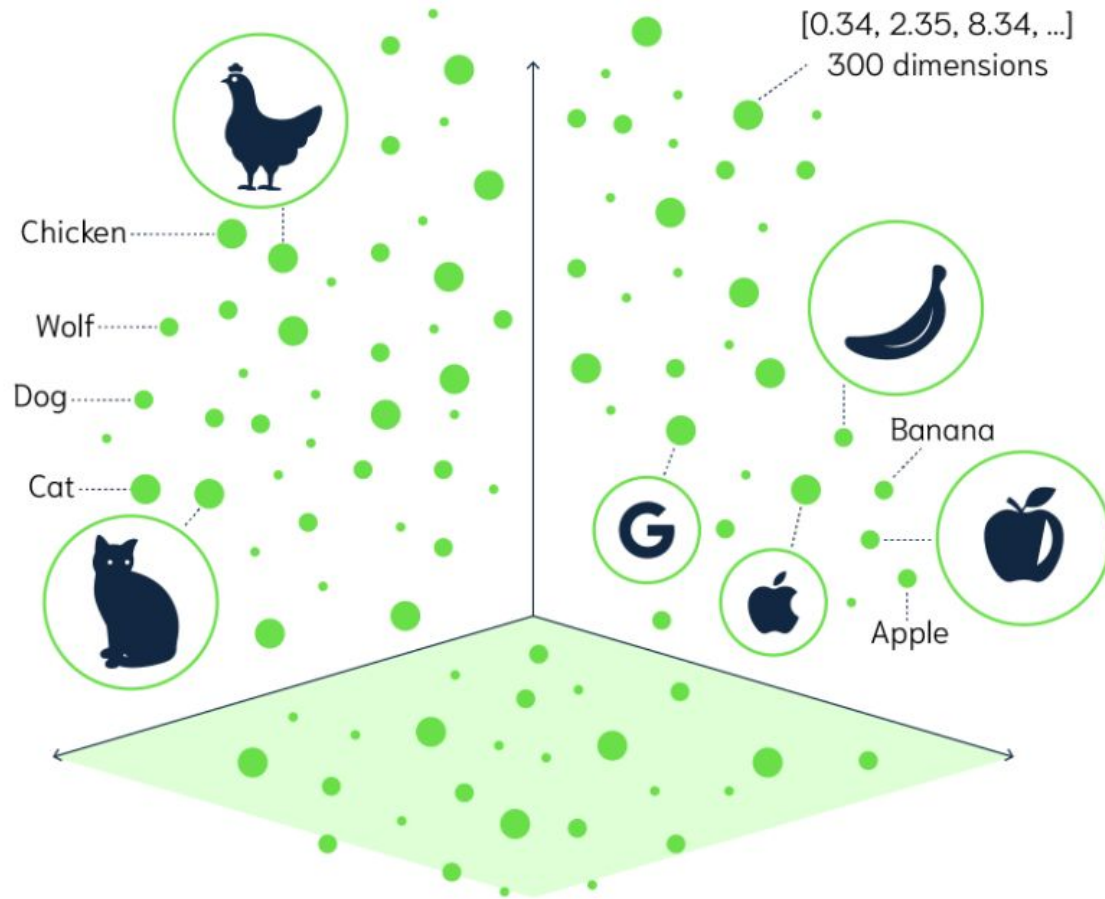


Fast Vector Query Processing for Large Datasets Beyond GPU Memory with Reordered Pipelining

Zili Zhang, Fangyue Liu, Gang Huang, Xuanzhe Liu, and Xin Jin

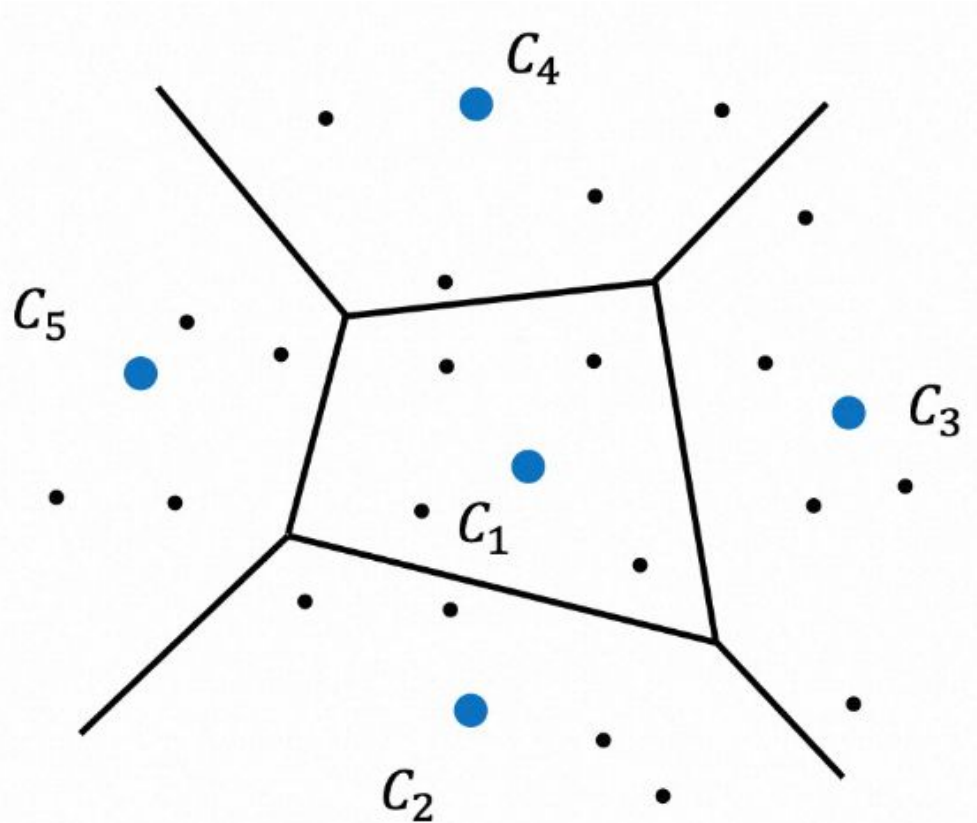
NSDI 2024

Background: Vector Query

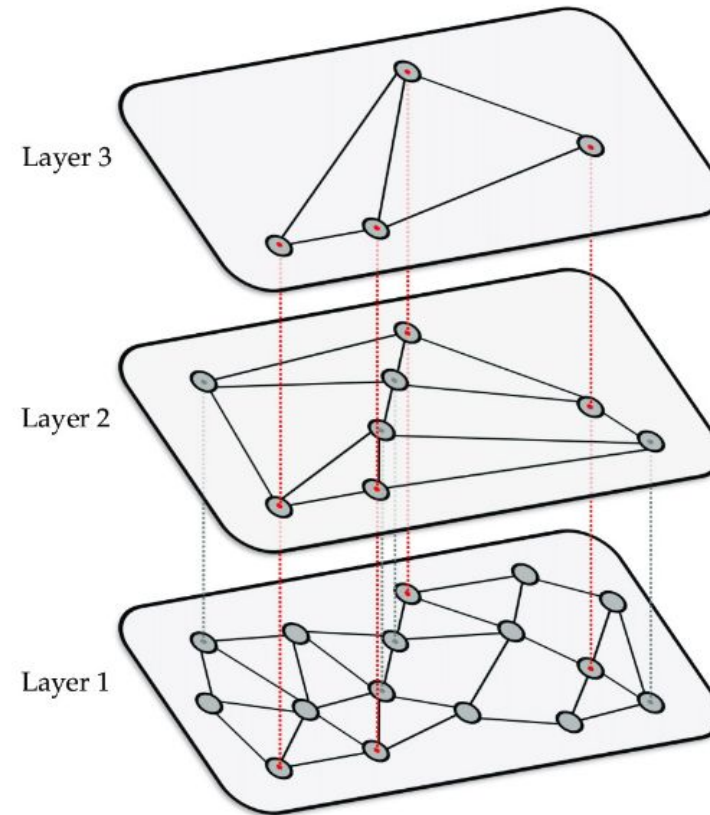


Background: Vector Indexes

IVF Index

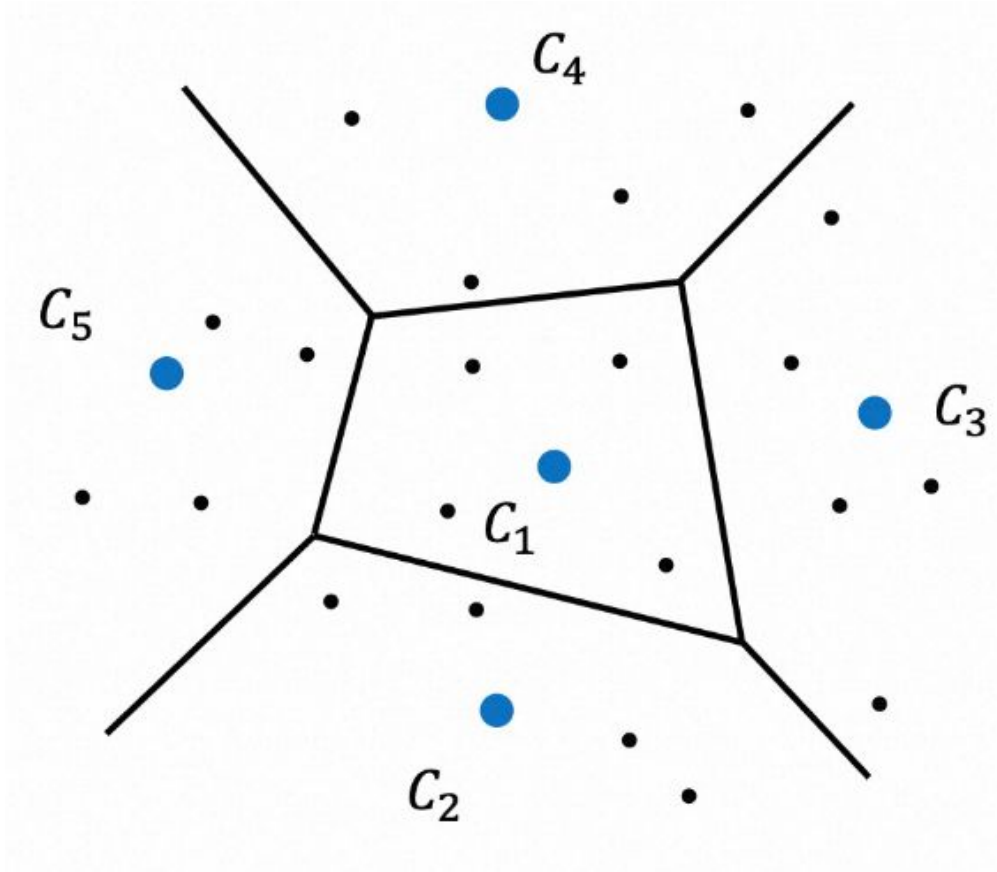


Graph Index



Background: IVF

IVF Index



IVF has:

- Simpler computation pattern
- Less memory footprint

Motivation: Where Do We Store Billion-Scale Vector Indexes?

Dataset	Dimensions	Database Vectors	Query Vectors	Memory Footprint
SIFT1B [25]	128	1B	10K	480 GB
DEEP1B [24]	96	1B	10K	361 GB
TEXT1B [24]	200	1B	100K	748 GB
SIFT40M [25]	128	40M	10K	31 GB
DEEP50M [24]	96	50M	10K	29 GB
TEXT30M [24]	200	30M	100K	32 GB
SIFT10M [25]	128	10M	10K	9 GB

Datasets used in evaluation

Component	Memory Type	Capacity
CPU (Host)	DDR4 RAM	1 TB
GPU	HBM2e VRAM	80 GB

Motivation: Where Do We Store Billion-Scale Vector Indexes?

Billion Scale Vectors **fits** in CPU memory

Dataset	Dimensions	Database Vectors	Query Vectors	Memory Footprint
SIFT1B [25]	128	1B	10K	480 GB
DEEP1B [24]	96	1B	10K	361 GB
TEXT1B [24]	200	1B	100K	748 GB
SIFT40M [25]	128	40M	10K	31 GB
DEEP50M [24]	96	50M	10K	29 GB
TEXT30M [24]	200	30M	100K	32 GB
SIFT10M [25]	128	10M	10K	9 GB

Datasets used in evaluation

Component	Memory Type	Capacity
CPU (Host)	DDR4 RAM	1 TB
GPU	HBM2e VRAM	80 GB

Motivation: Where Do We Store Billion-Scale Vector Indexes?

Billion Scale Vectors **does not fit** in GPU memory !

Dataset	Dimensions	Database Vectors	Query Vectors	Memory Footprint
SIFT1B [25]	128	1B	10K	480 GB
DEEP1B [24]	96	1B	10K	361 GB
TEXT1B [24]	200	1B	100K	748 GB
SIFT40M [25]	128	40M	10K	31 GB
DEEP50M [24]	96	50M	10K	29 GB
TEXT30M [24]	200	30M	100K	32 GB
SIFT10M [25]	128	10M	10K	9 GB

Datasets used in evaluation

Component	Memory Type	Capacity
CPU (Host)	DDR4 RAM	1 TB
GPU	HBM2e VRAM	80 GB

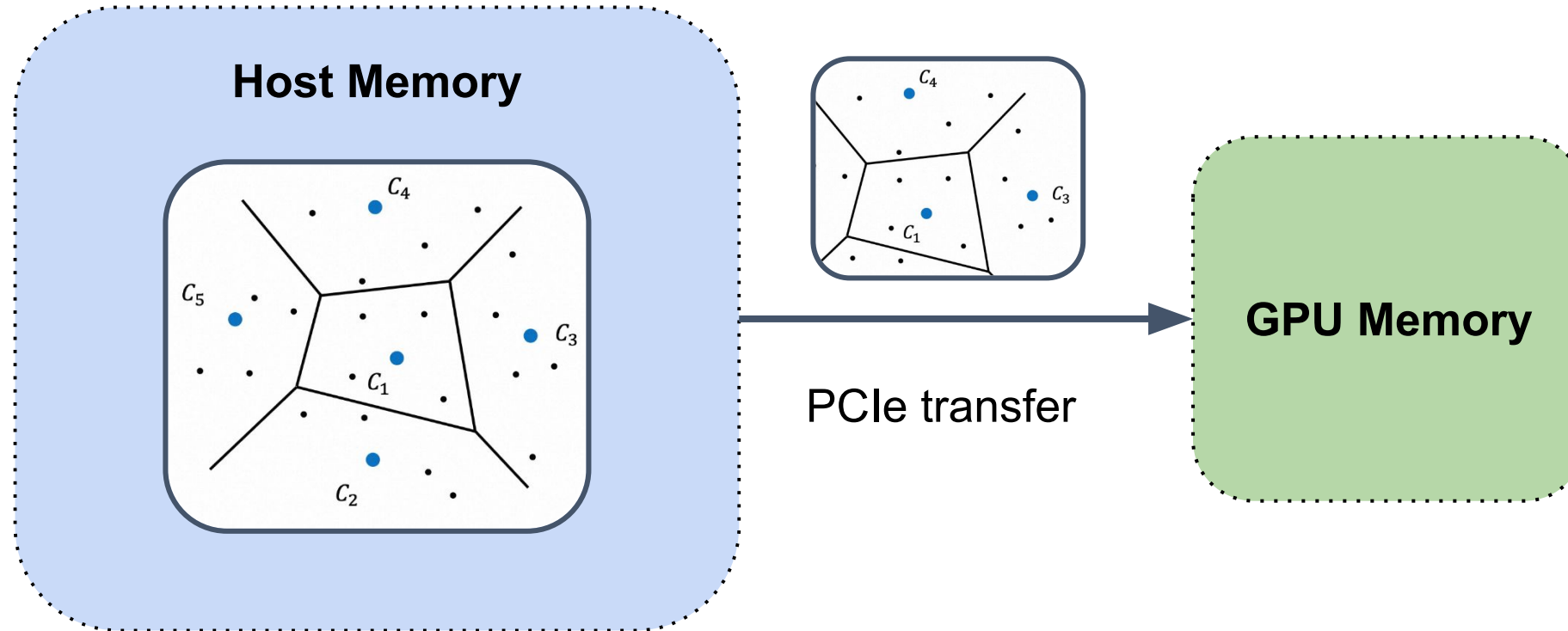
Motivation: Where Do We Store Billion-Scale Vector Indexes?

Billion Scale Vectors fails to **fit** in GPU memory !

Dataset	Dimensions	Database Vectors	Query Vectors	Memory Footprint	Component	Memory	Capacity
SIFT1B					Limited GPU Memory makes storing Billion Scale Vectors infeasible!		
DEEP1B							
TEXT1B							
SIFT40M							
DEEP50M							
TEXT30M [24]	200	30M	100K	32 GB	GPU	HBM2e VRAM	80 GB
SIFT10M [25]	128	10M	10K	9 GB			

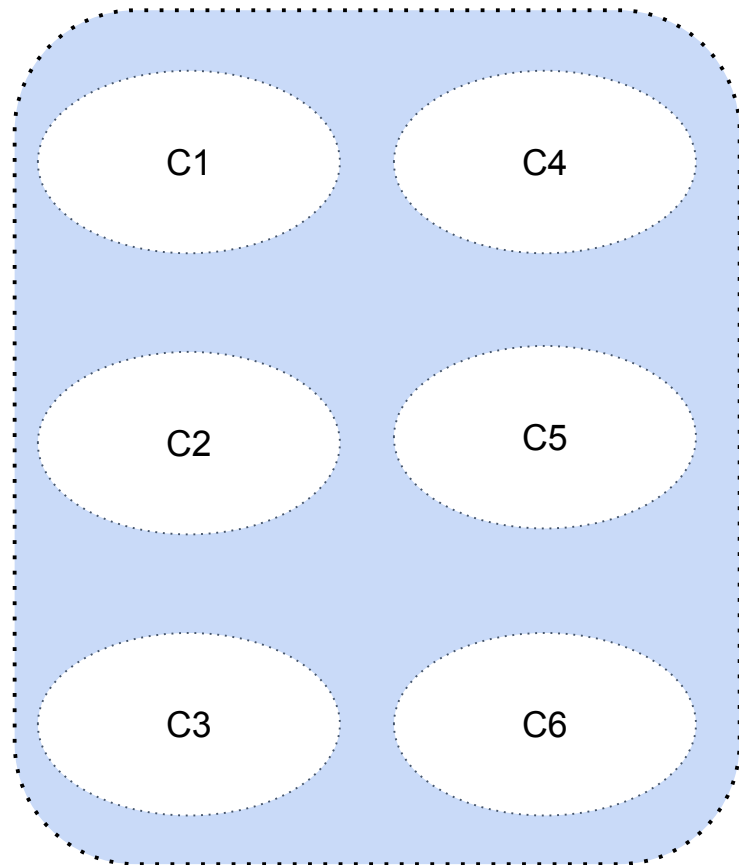
Datasets used in evaluation

Motivation: Host to GPU Transfer

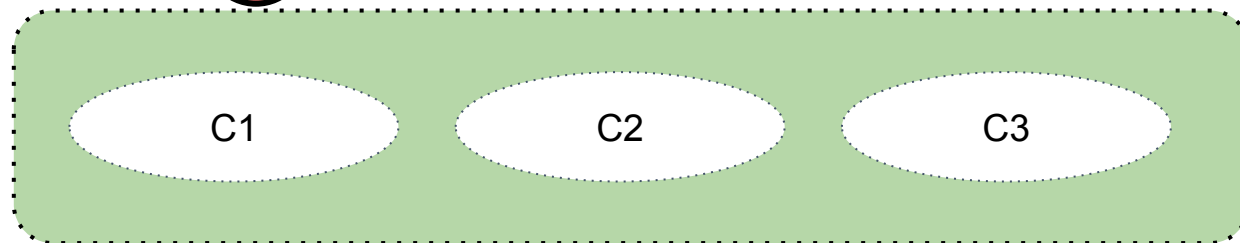


Challenges: Redundant Transmissions

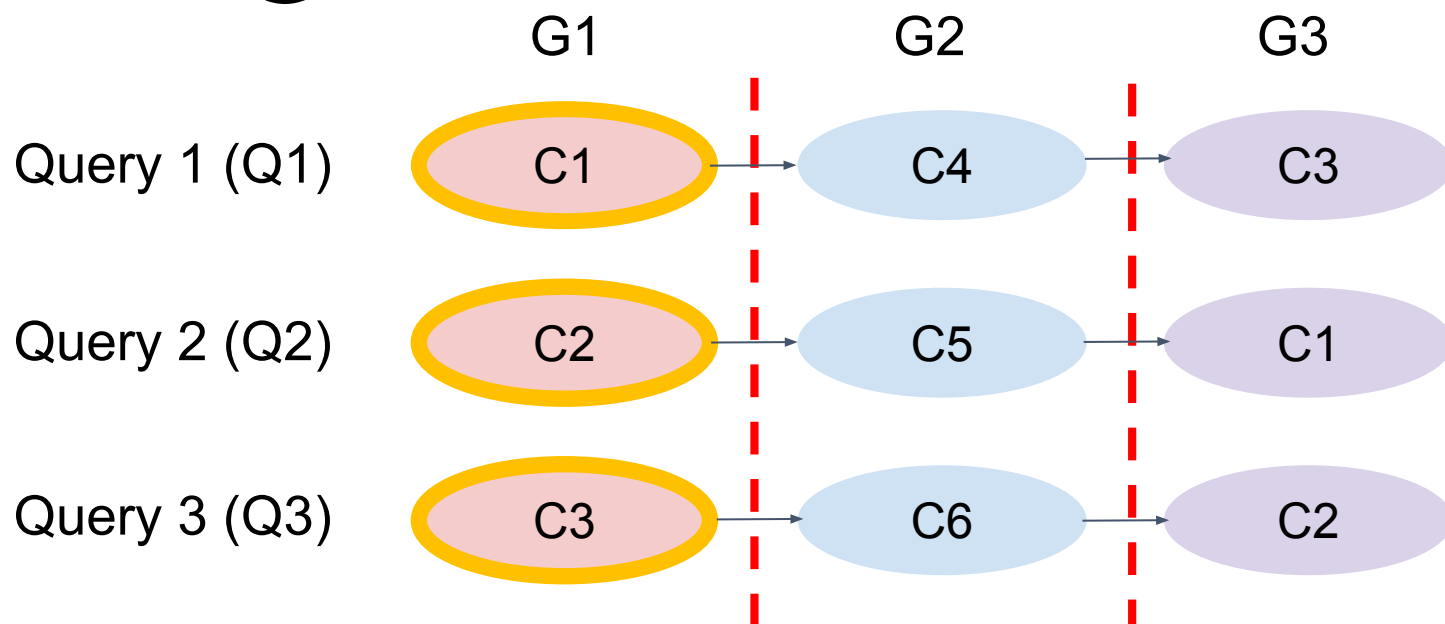
1 CPU Memory: All IVF index Clusters



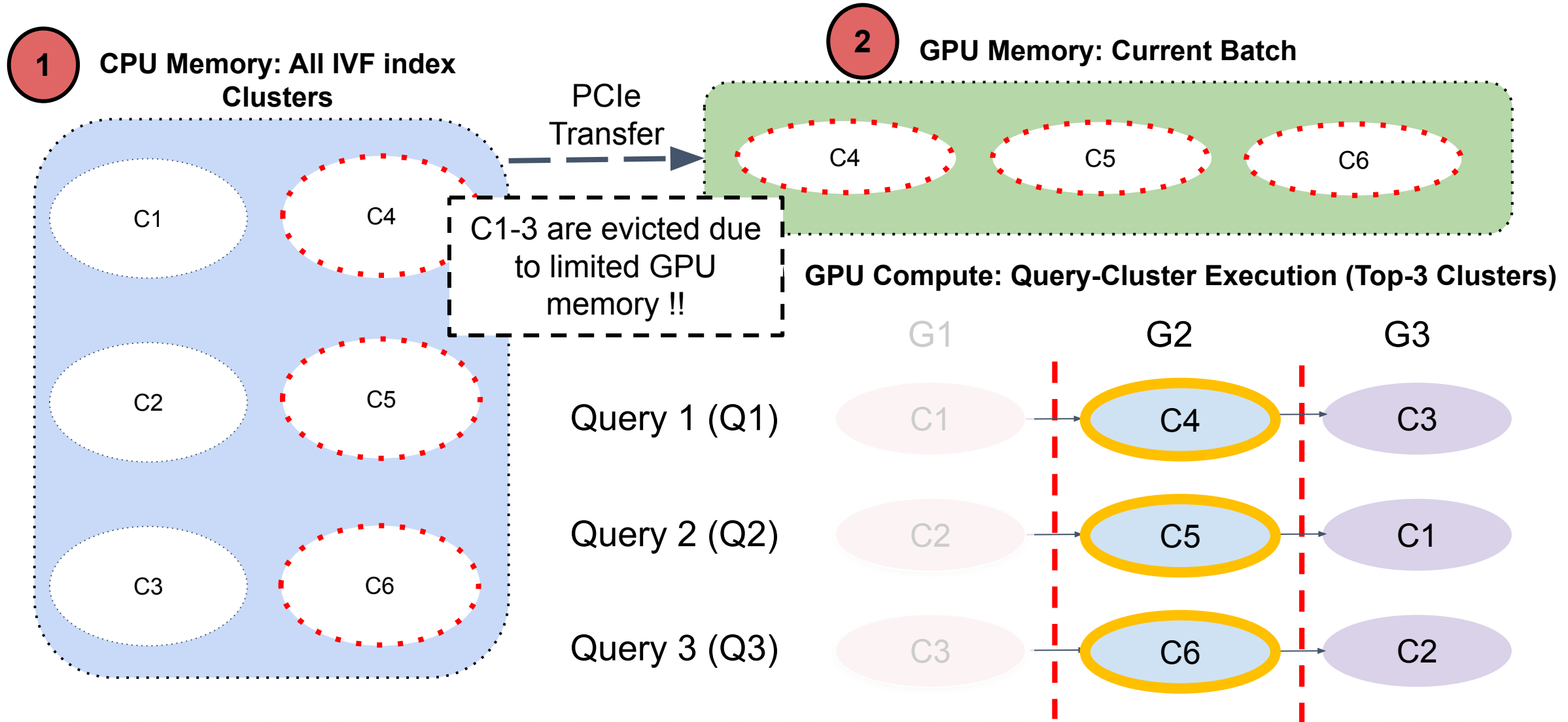
2 GPU Memory: Current Batch



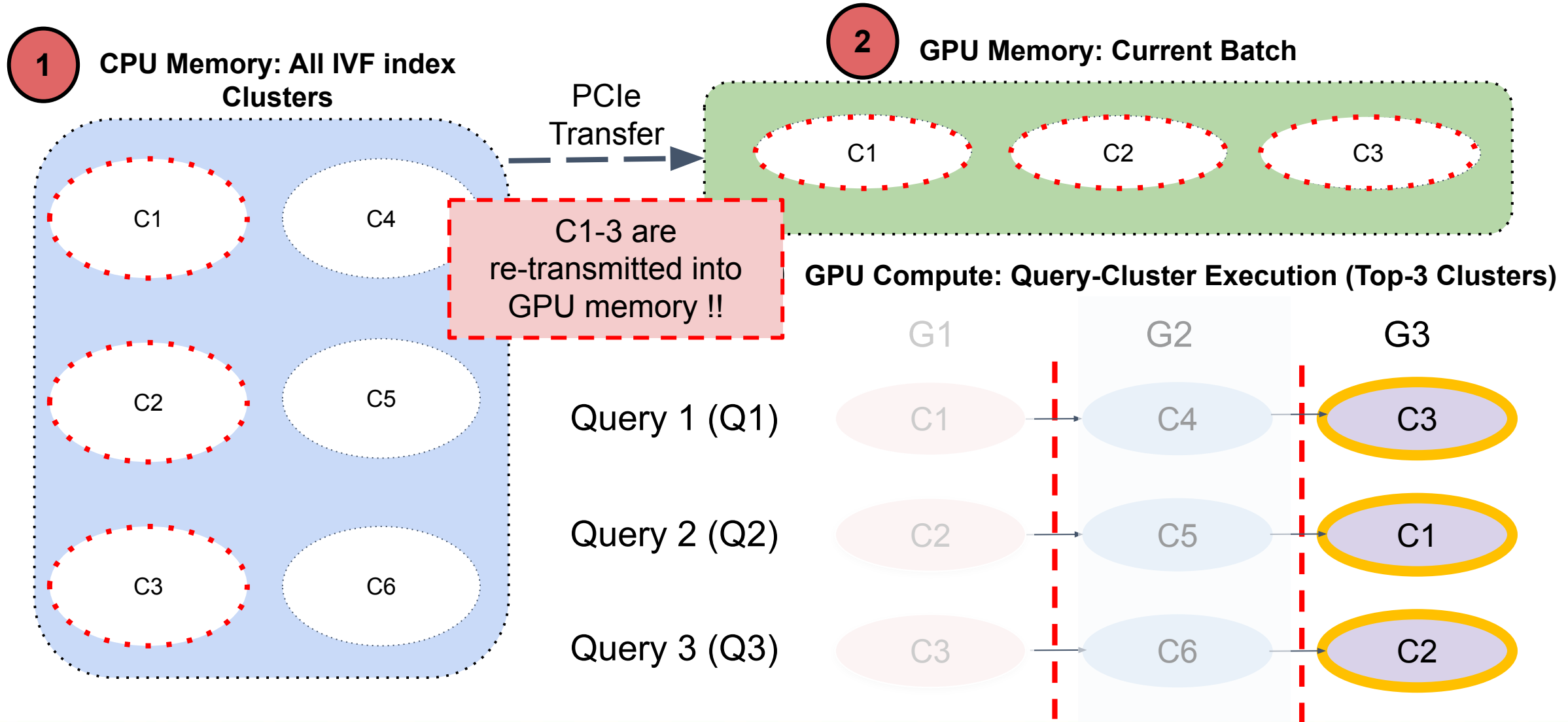
3 GPU Compute: Query-Cluster Execution (Top-3 Clusters)



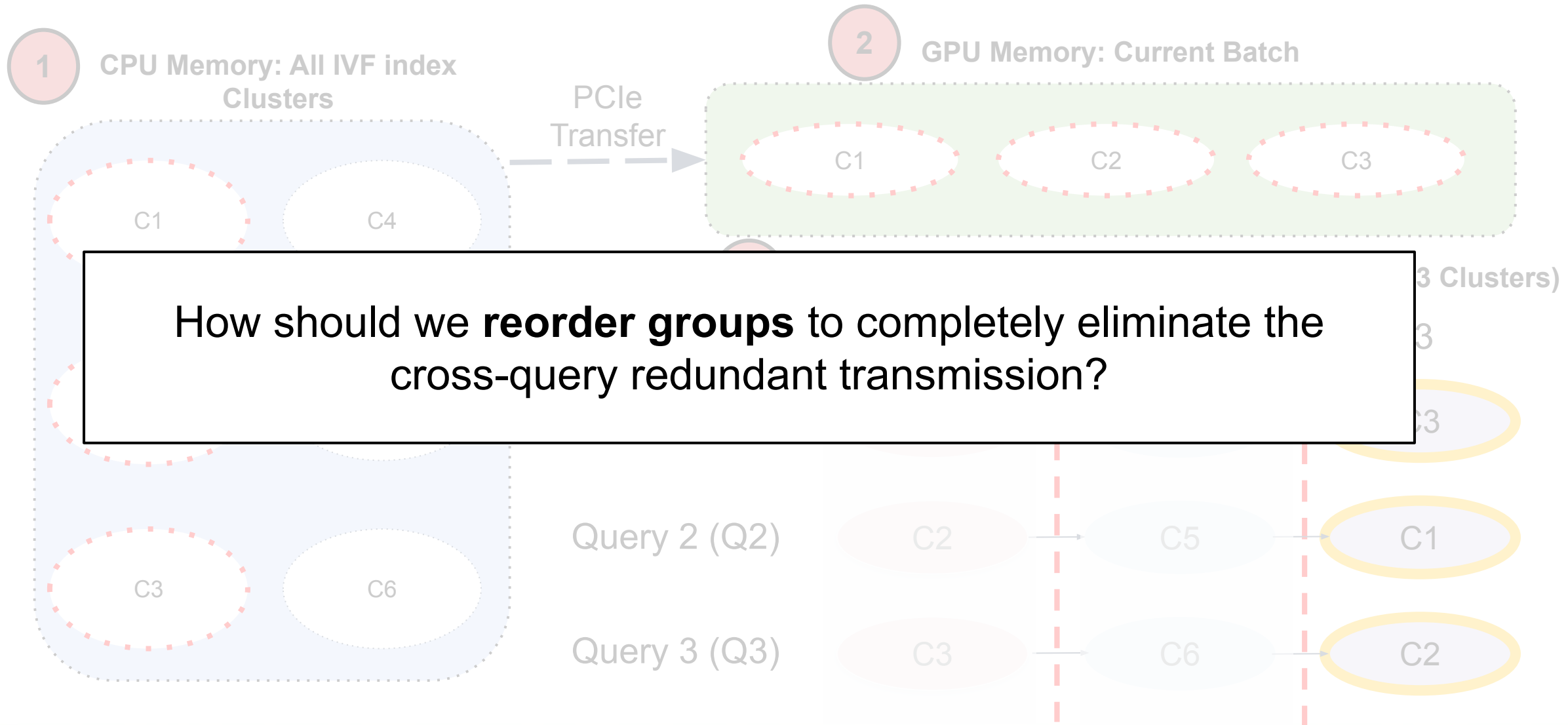
Challenges: Redundant Transmissions



Challenges: Redundant Transmissions

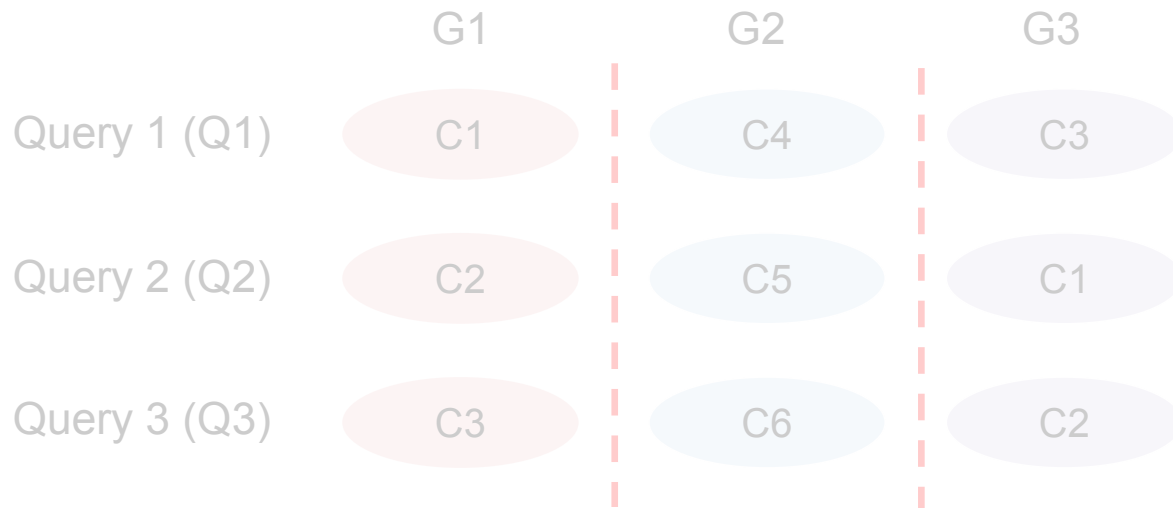


Challenges: Redundant Transmissions



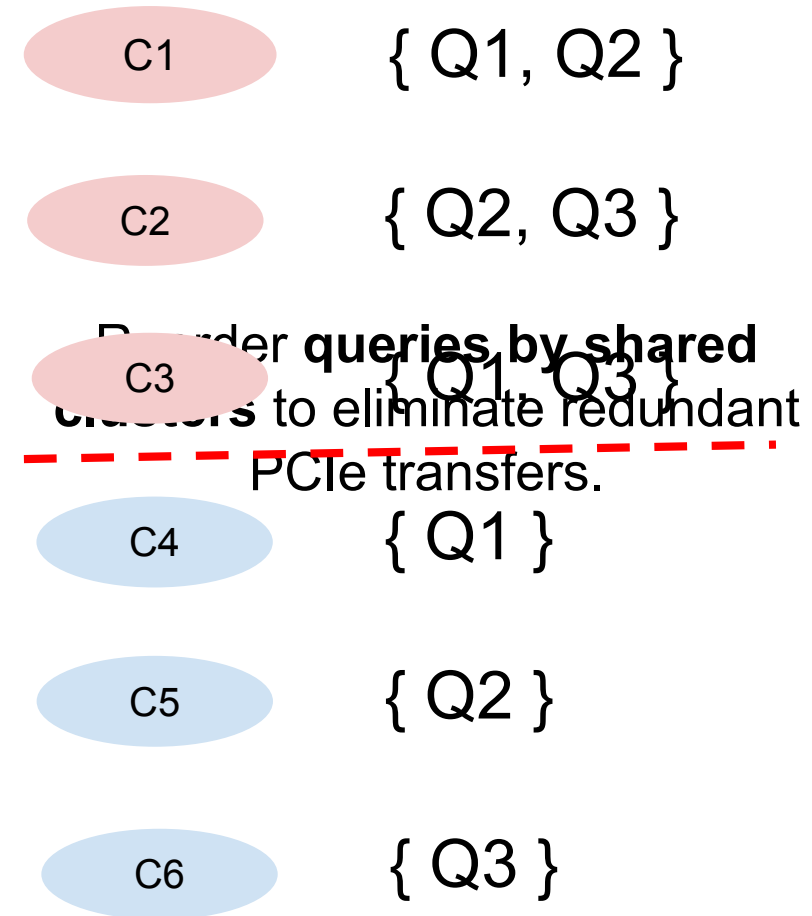
Solution: Cluster-Based Retrofitting

Query Centric view



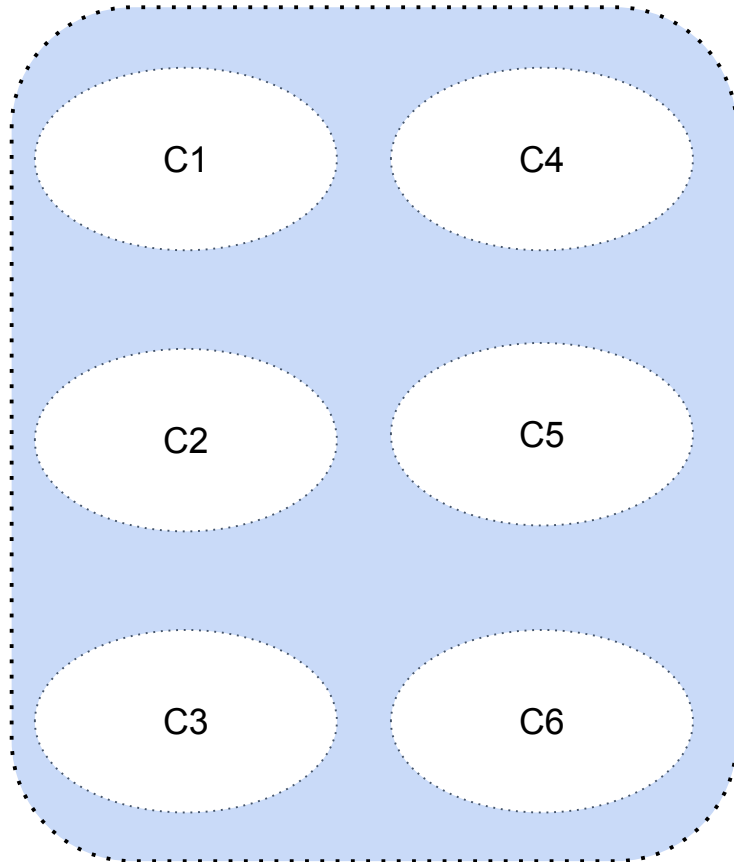
Reduced to 6 transmissions from CPU memory to GPU memory, one per cluster !

Cluster Centric view

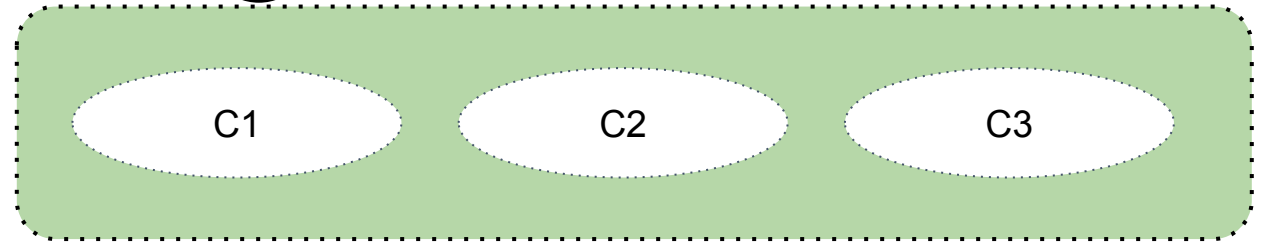


Solution: Cluster-Based Retrofitting

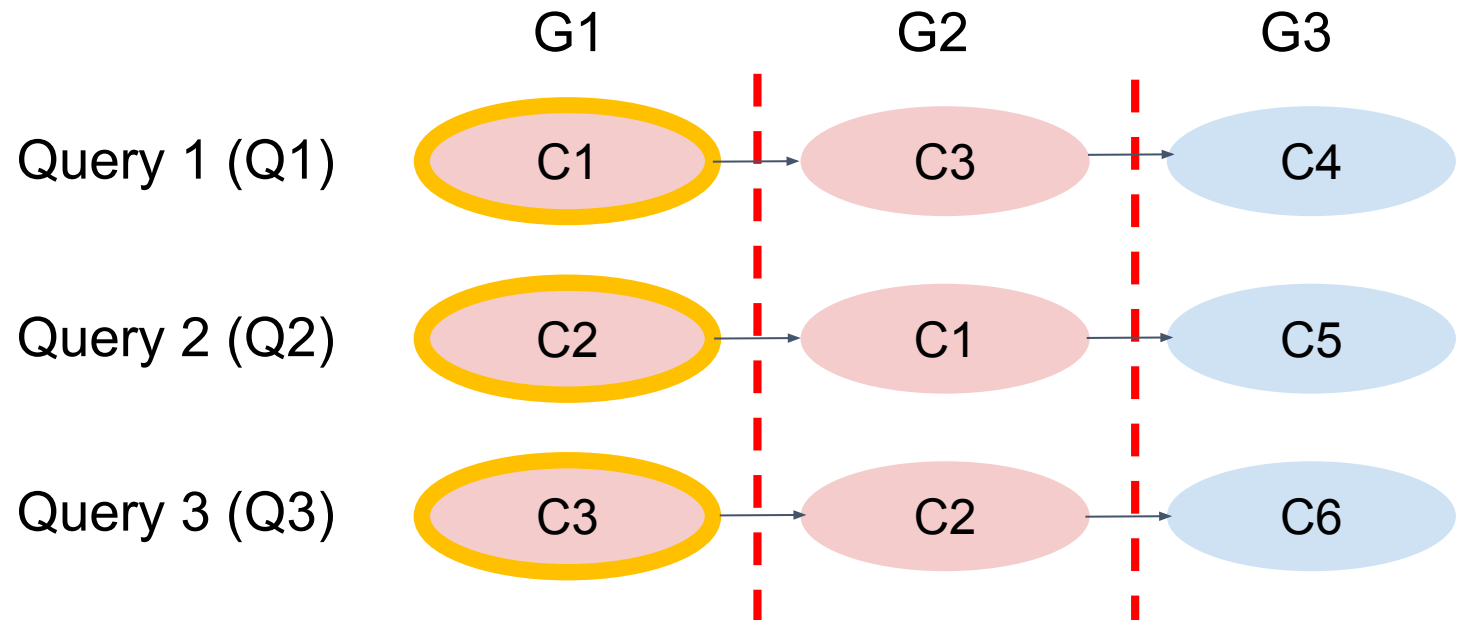
1 CPU Memory: All IVF index Clusters



2 GPU Memory: Current Batch

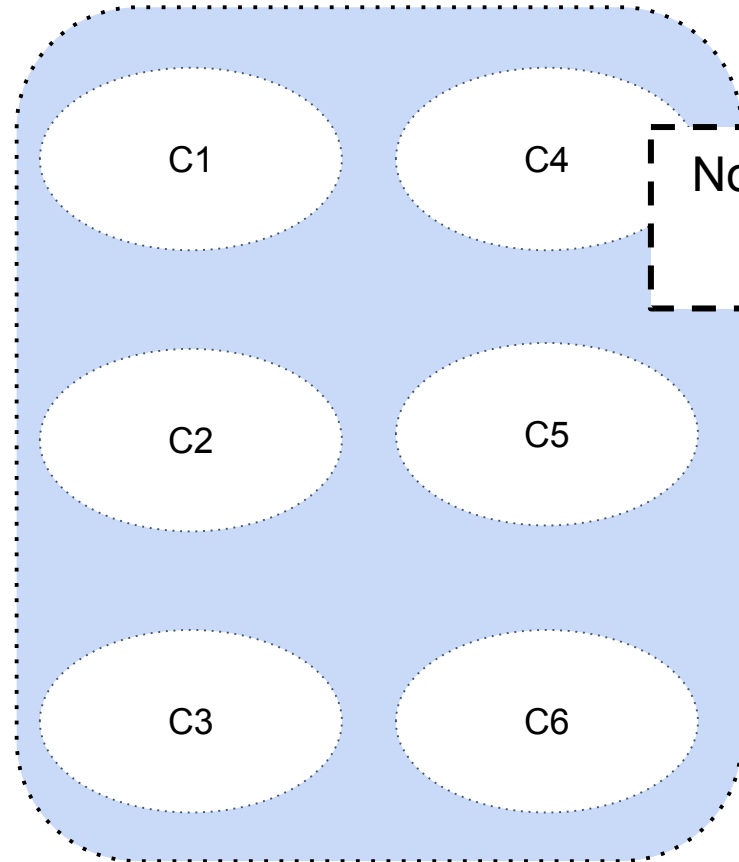


3 GPU Compute: Query-Cluster Execution (Top-3 Clusters)

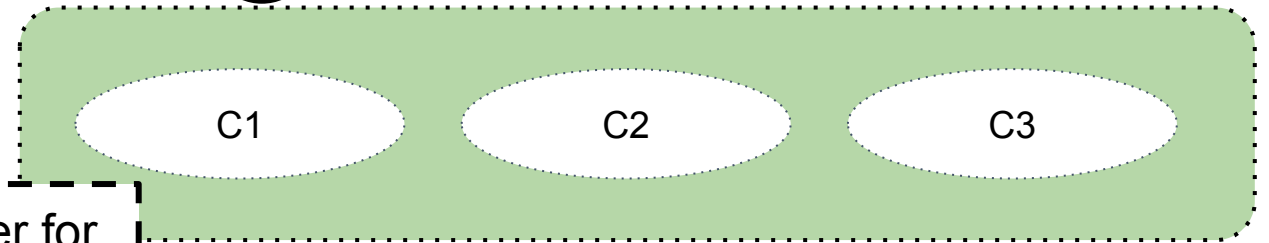


Solution: Cluster-Based Retrofitting

1 CPU Memory: All IVF index Clusters

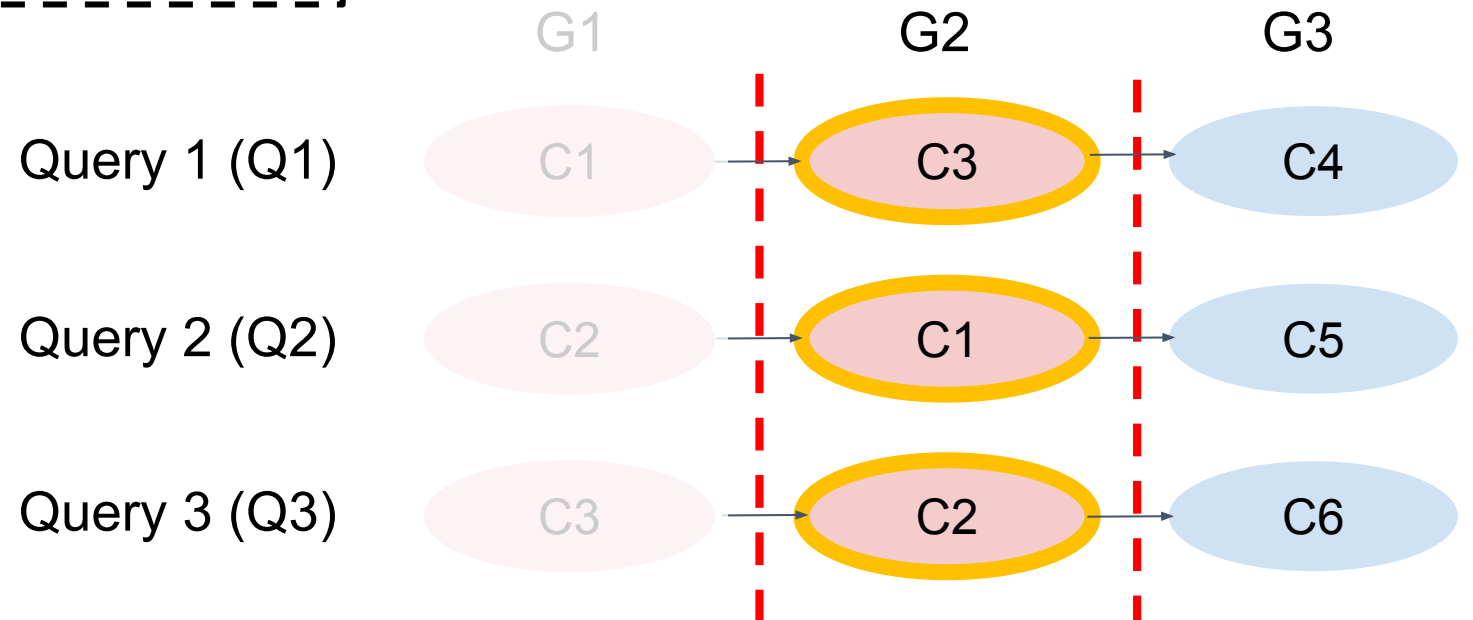


2 GPU Memory: Current Batch

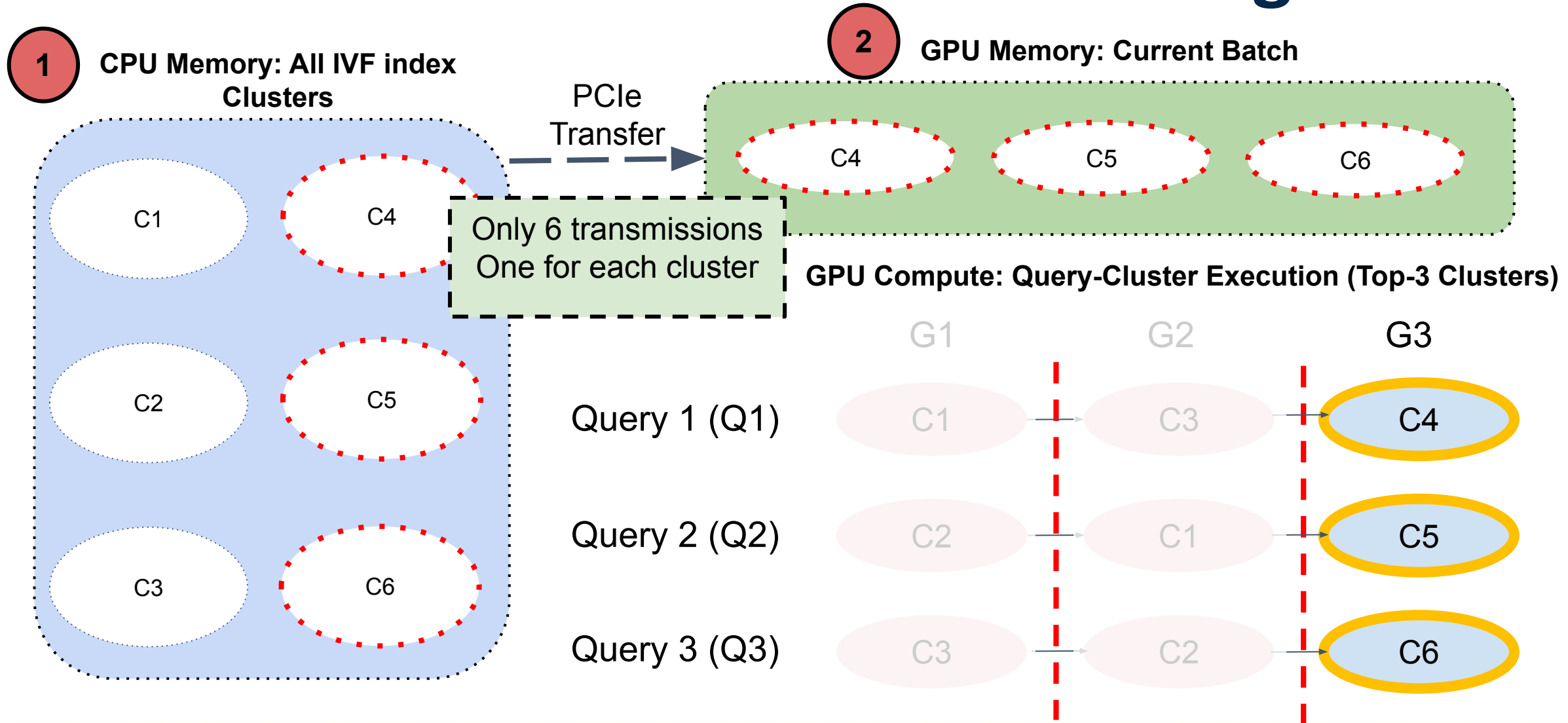


No PCIe transfer for
executing G2

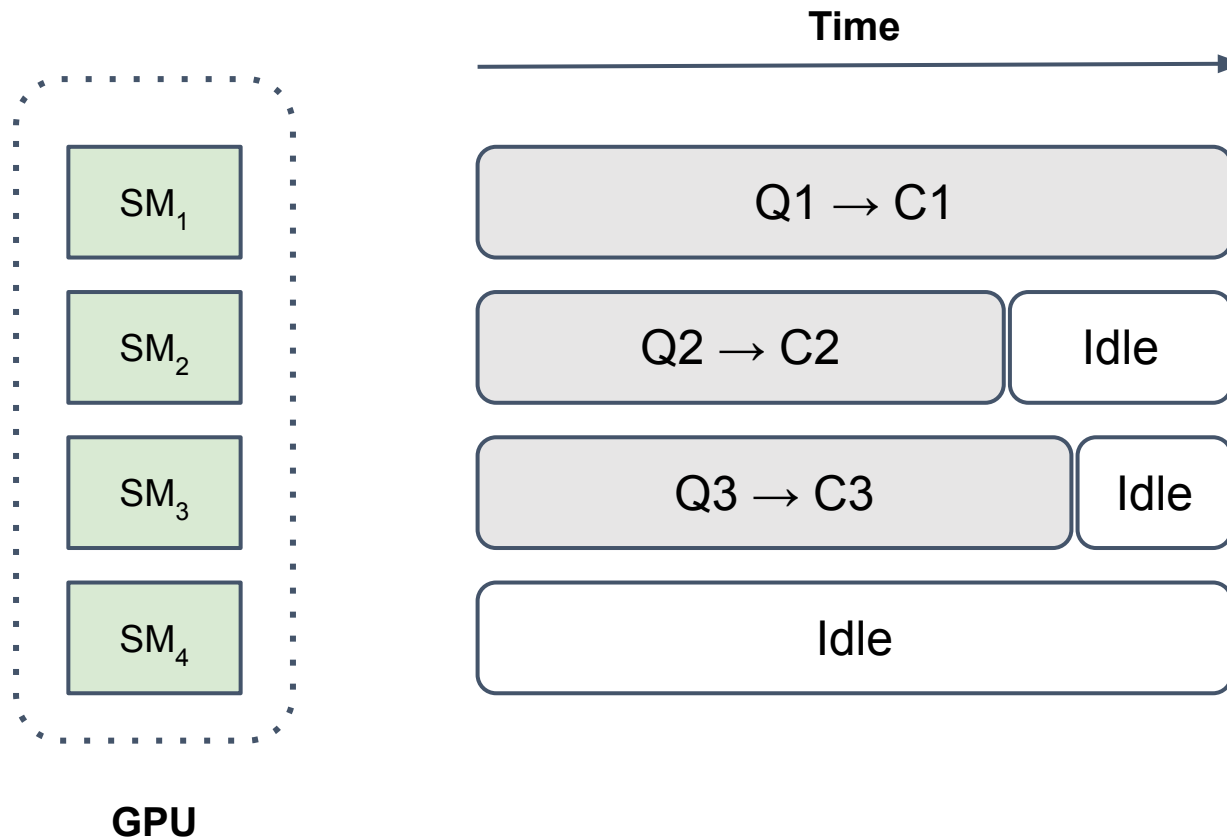
GPU Compute: Query-Cluster Execution (Top-3 Clusters)



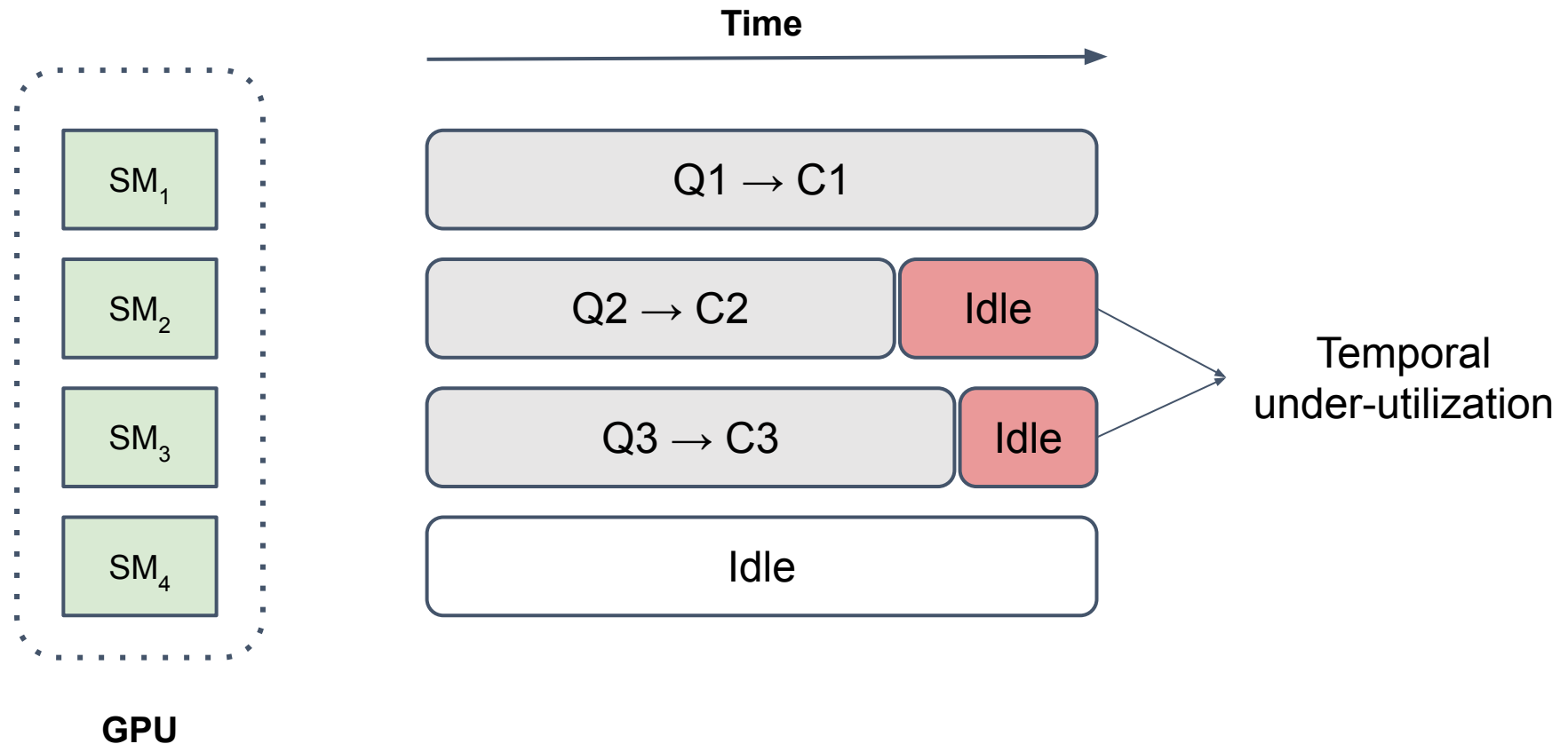
Solution: Cluster-Based Retrofitting



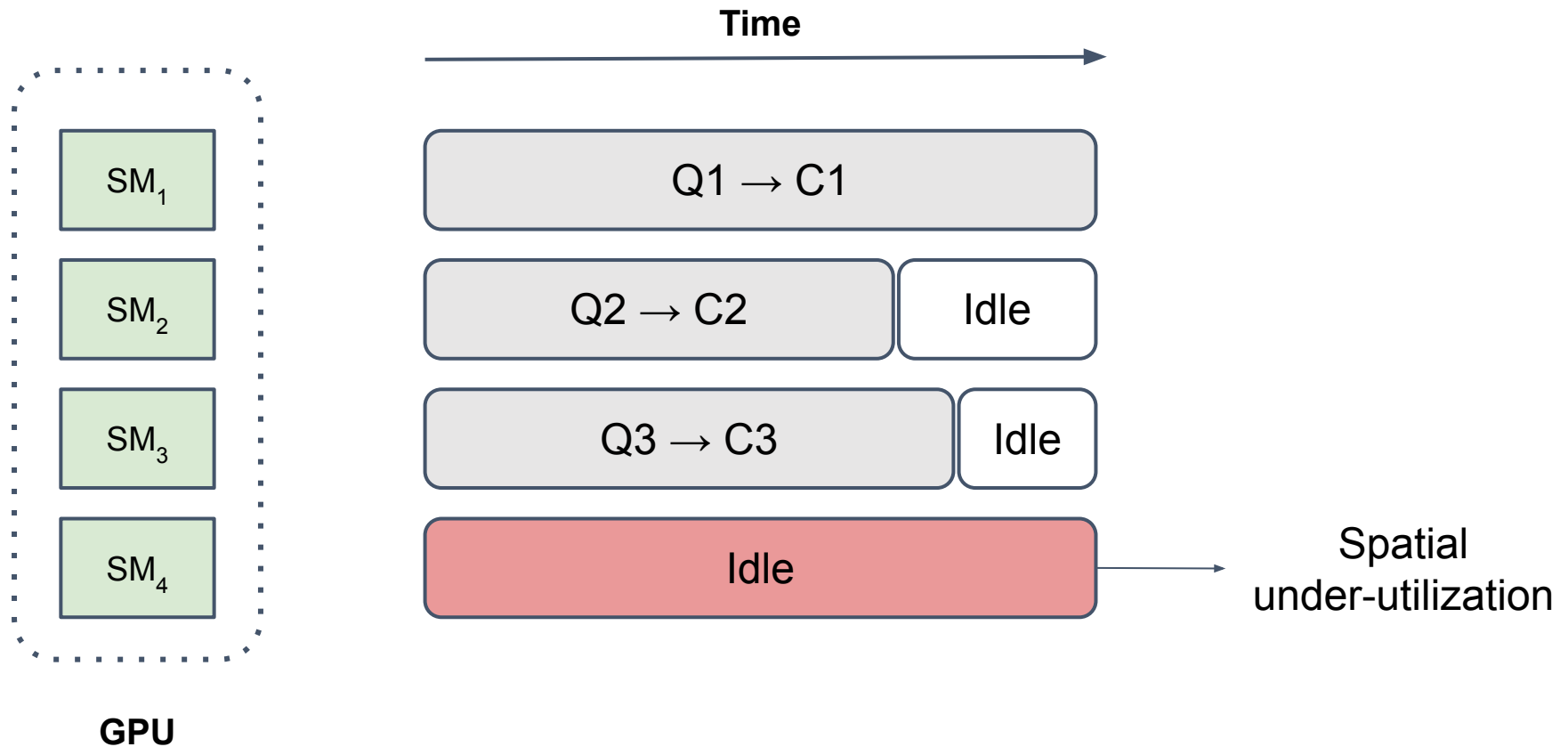
Challenges: Compute Under-utilization



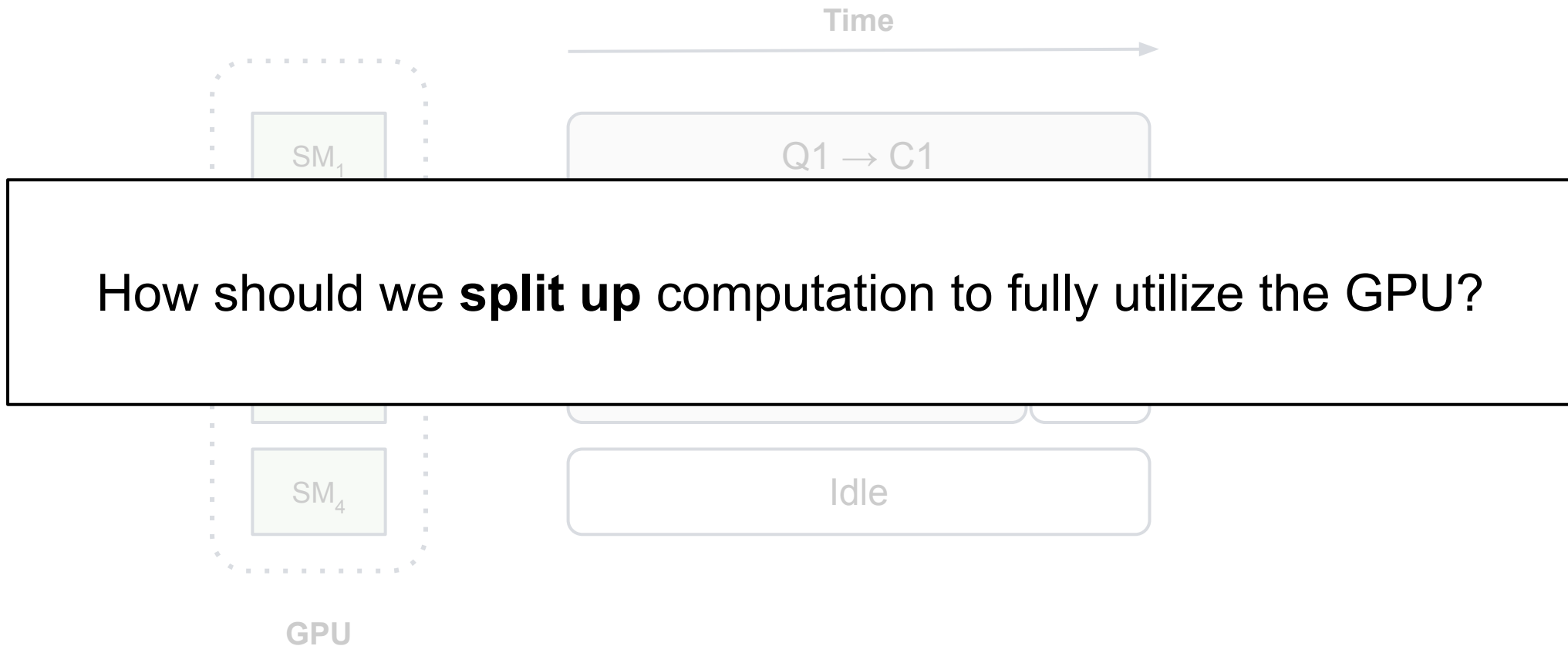
Challenges: Compute Under-utilization



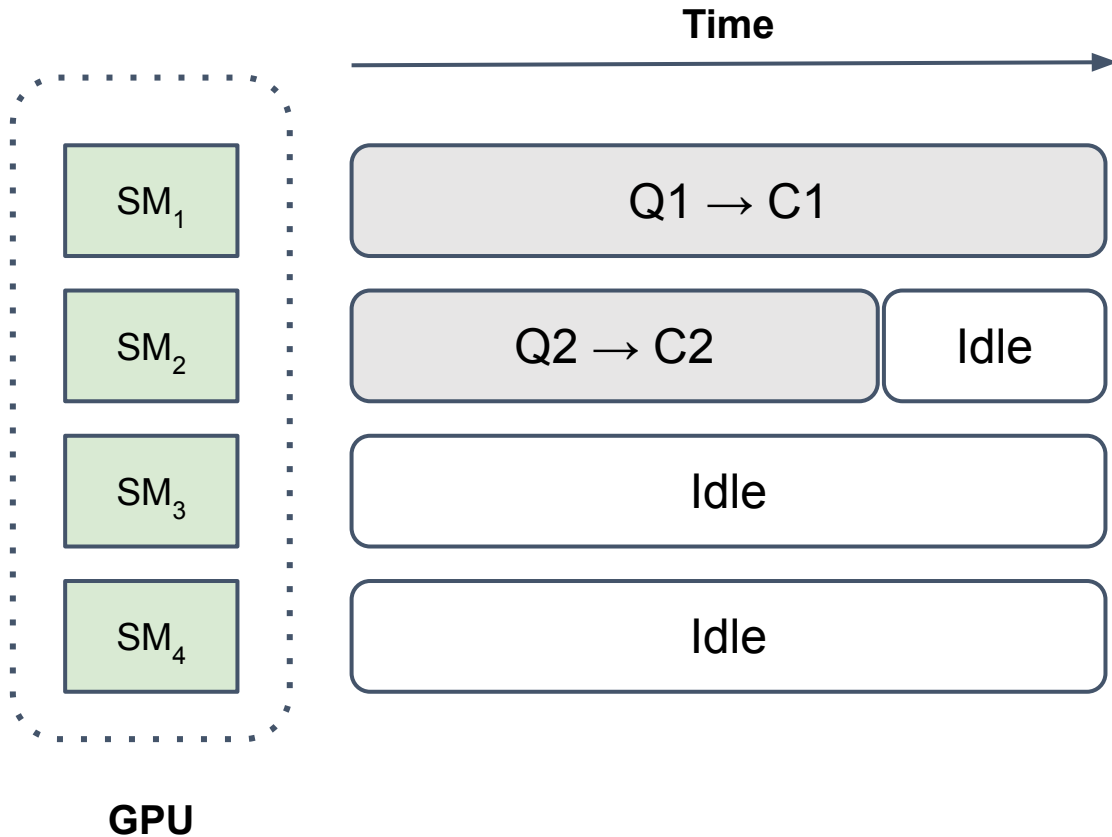
Challenges: Compute Under-utilization



Challenges: Compute Under-utilization

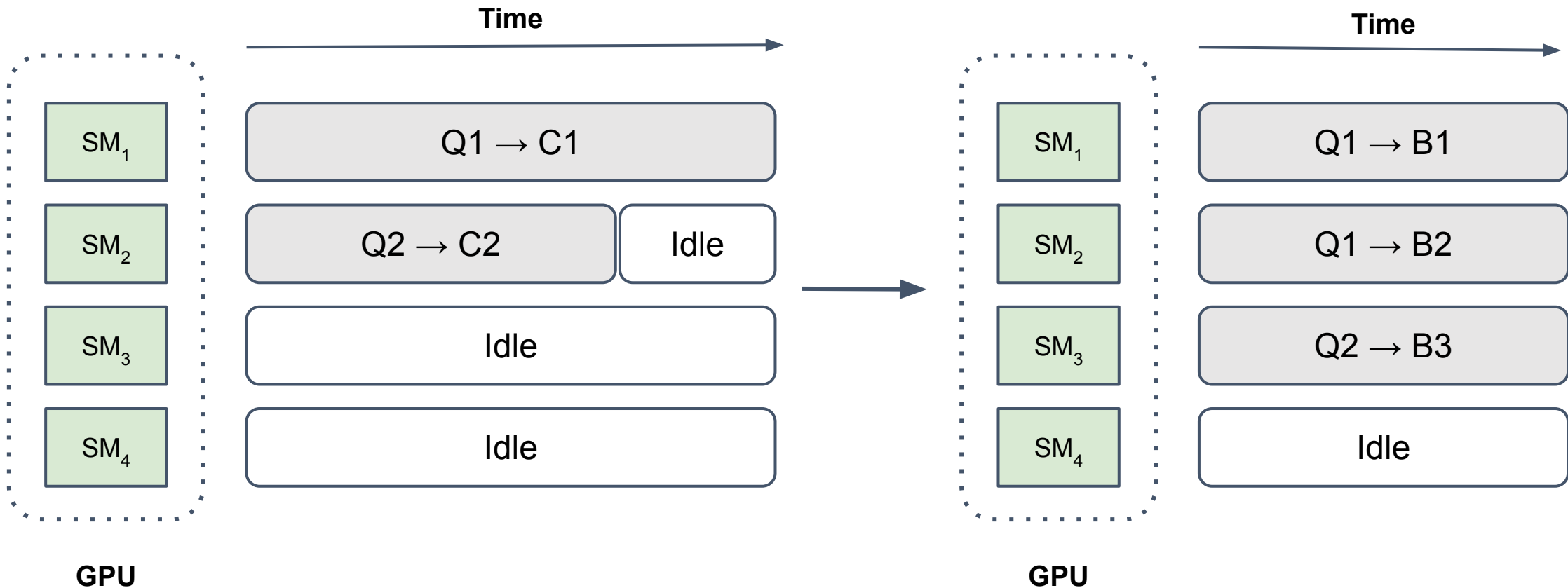


Solution: Cluster Balancing

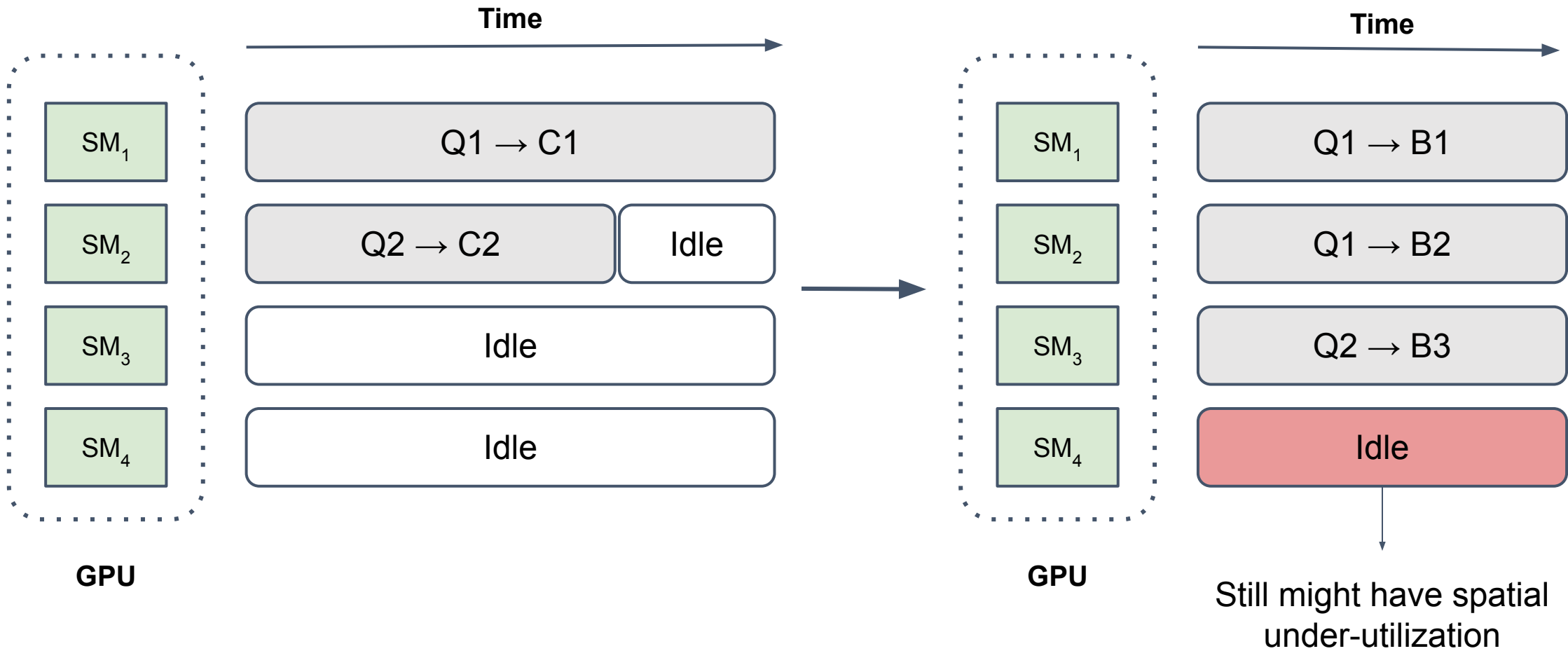


Split clusters up into (nearly)
equal-sized **balanced clusters**
offline

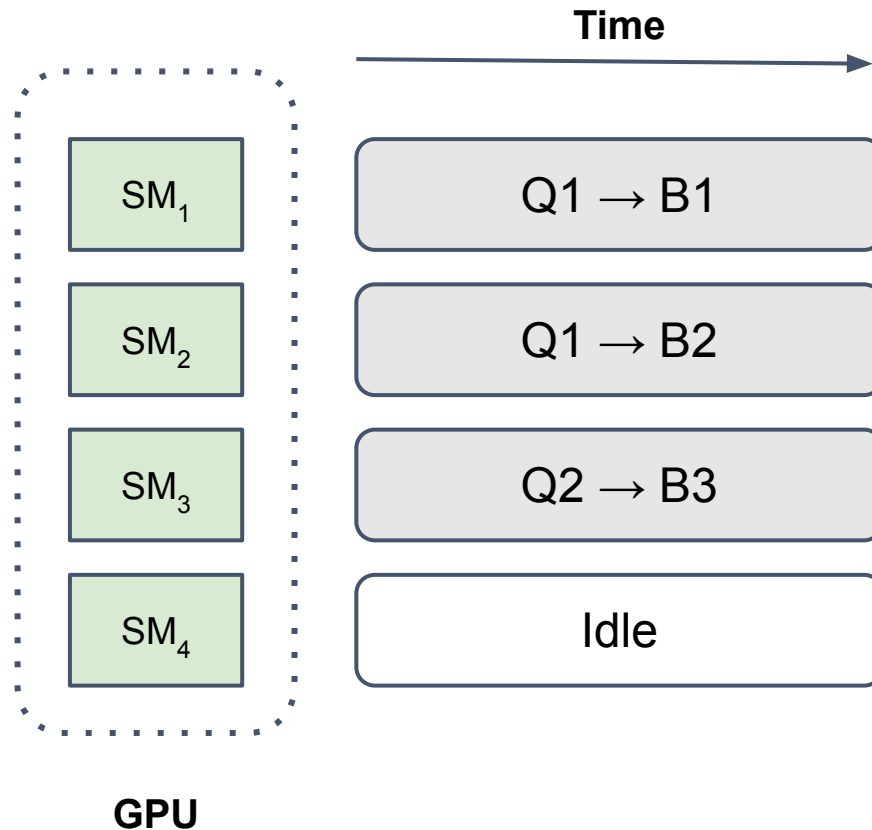
Solution: Cluster Balancing



Solution: Cluster Balancing

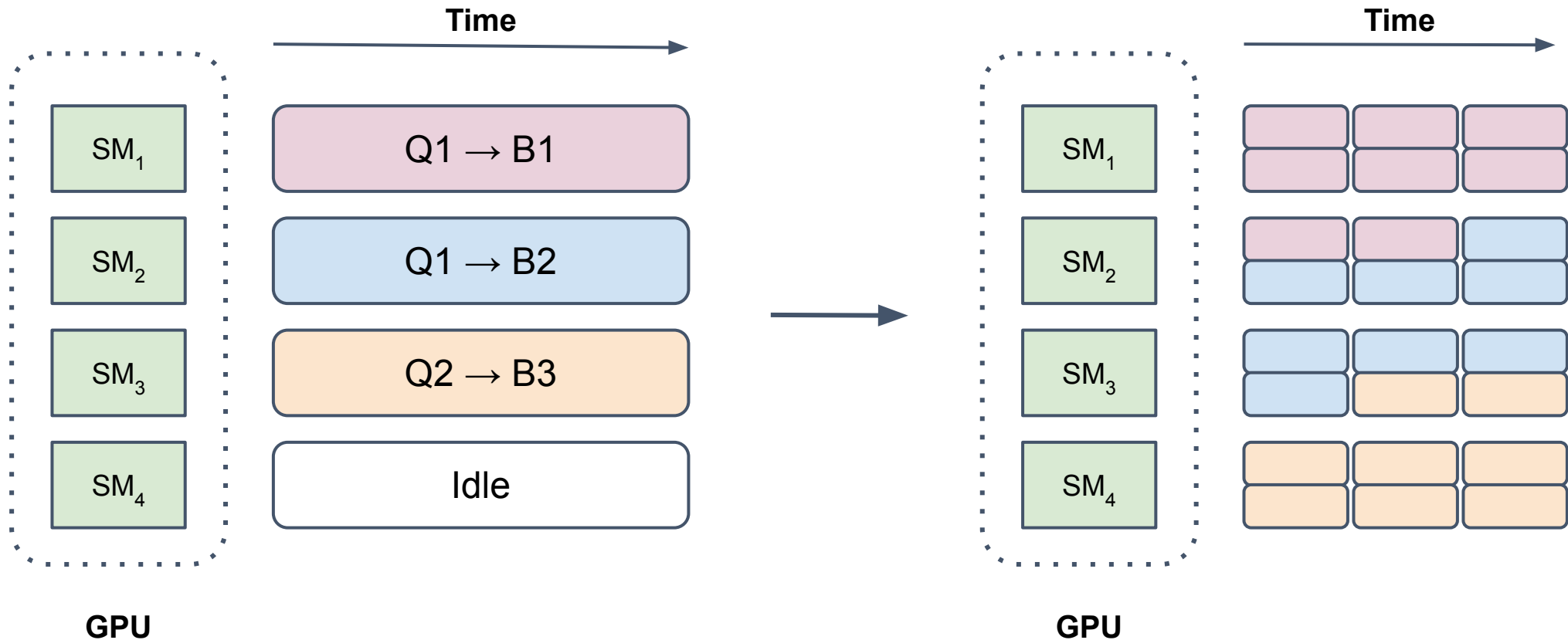


Solution: Dynamic Kernel Padding

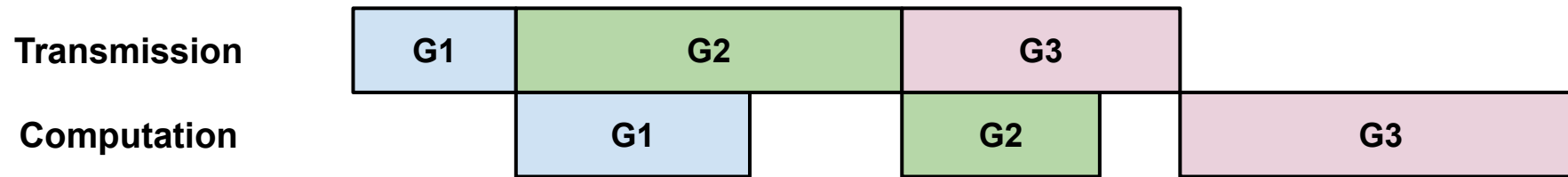


Divide thread blocks into smaller thread blocks **at runtime** based on query plan and no. of SMs

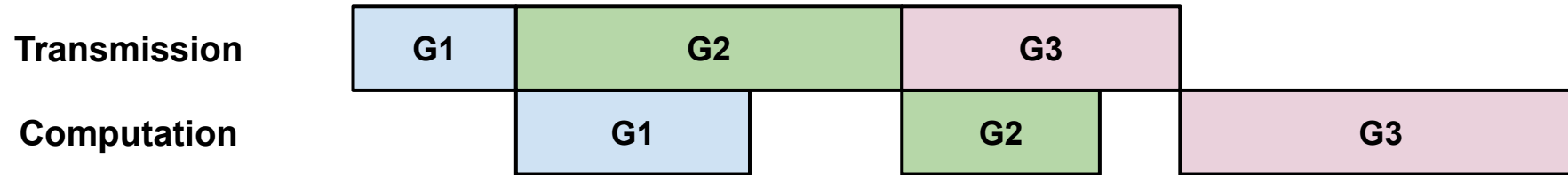
Solution: Dynamic Kernel Padding



Challenges: Pipeline Bubbles

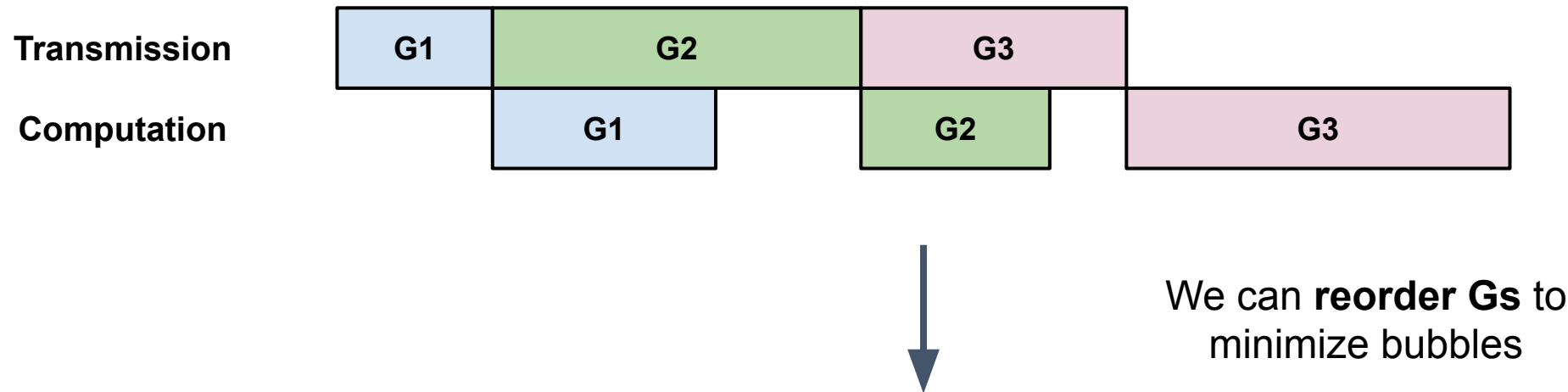


Challenges: Pipeline Bubbles

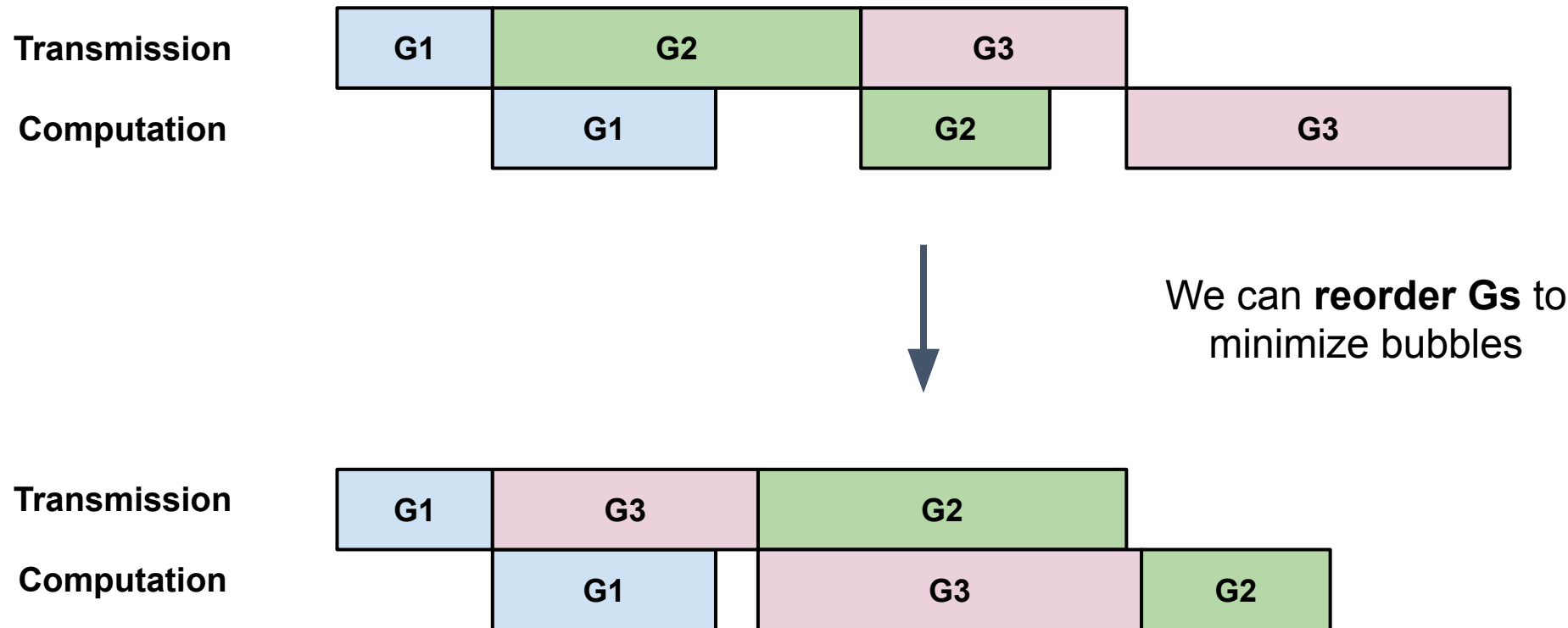


Too many bubbles!

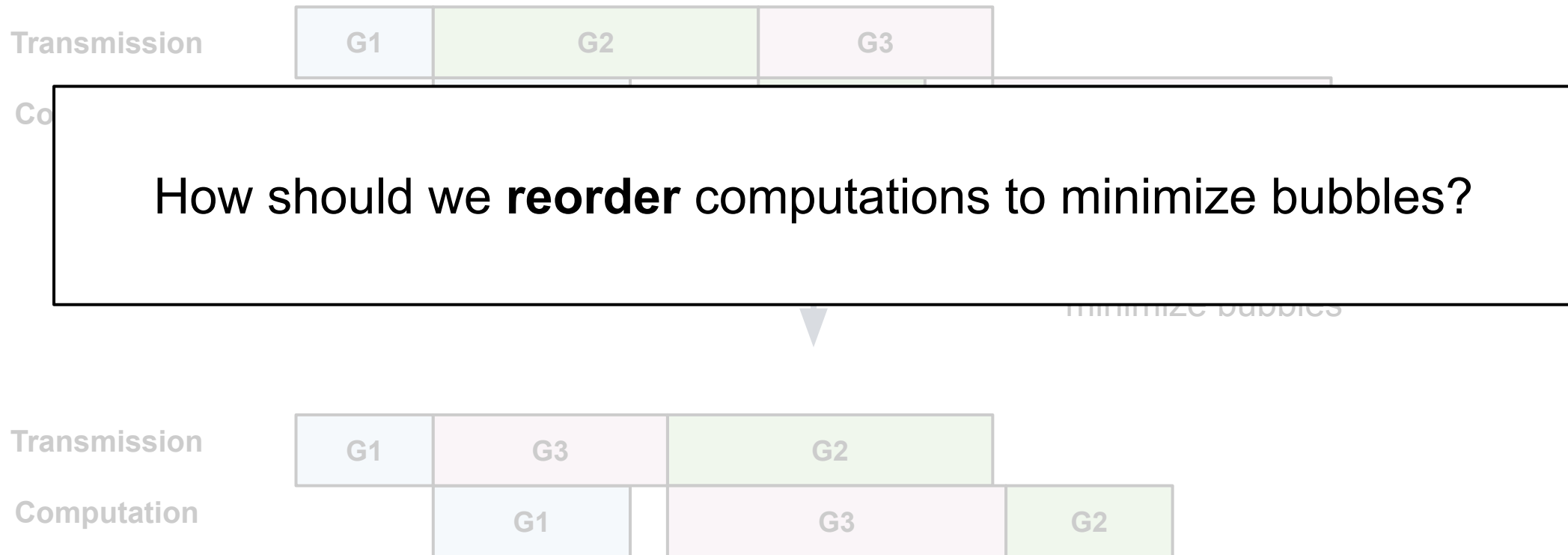
Challenges: Pipeline Bubbles



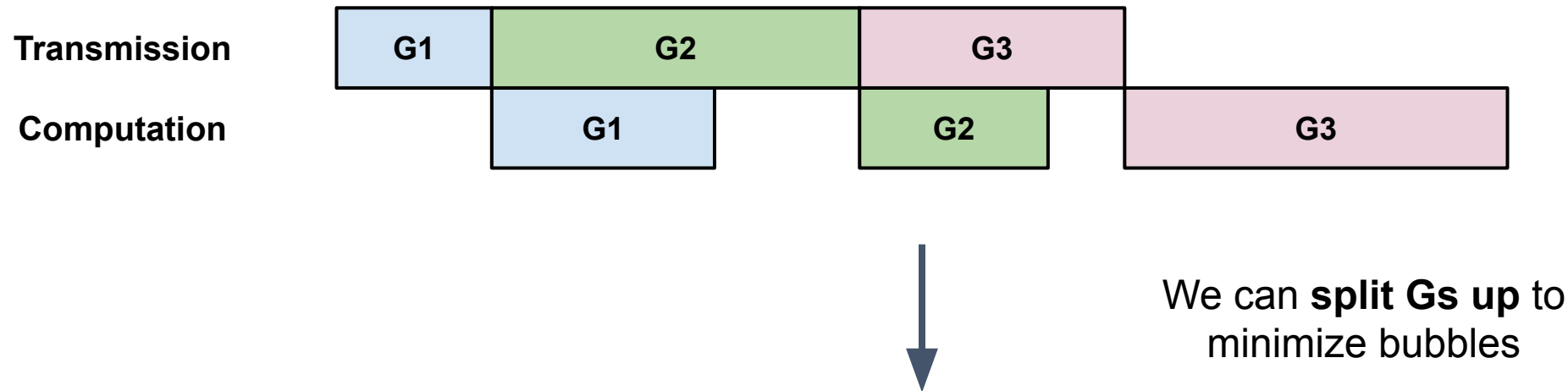
Challenges: Pipeline Bubbles



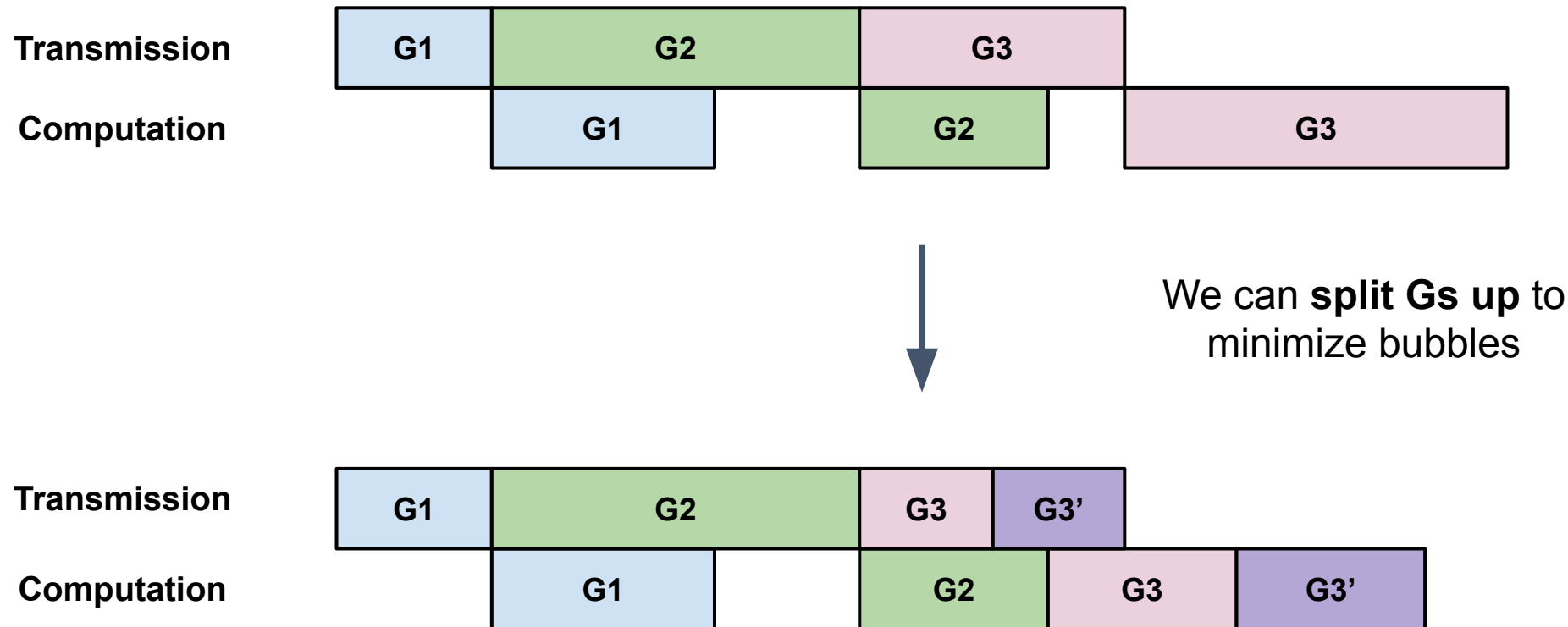
Challenges: Pipeline Bubbles



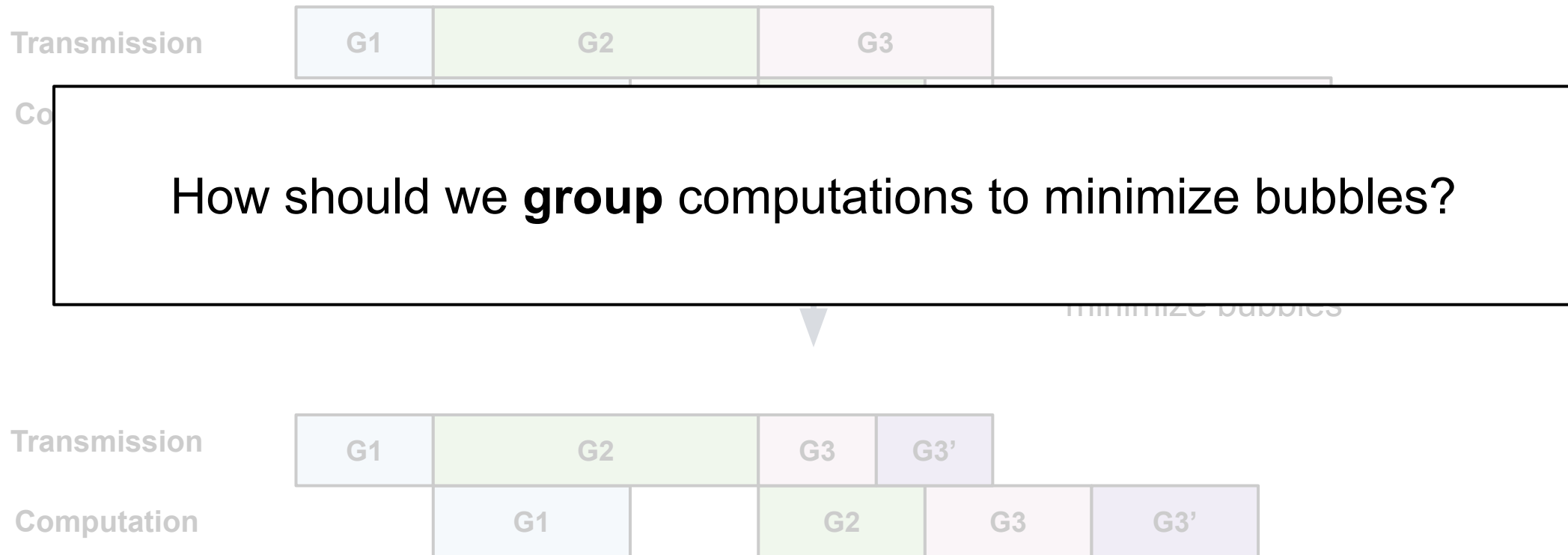
Challenges: Pipeline Bubbles



Challenges: Pipeline Bubbles



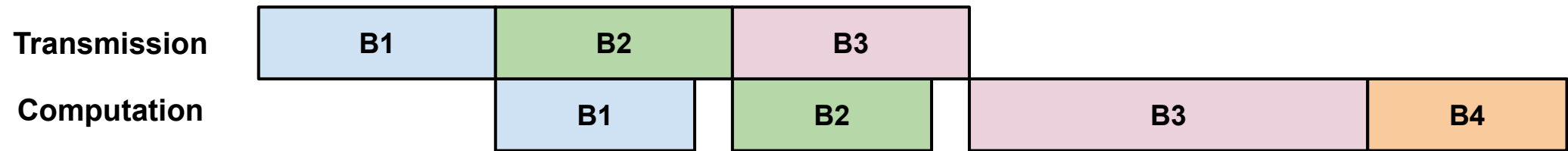
Challenges: Pipeline Bubbles



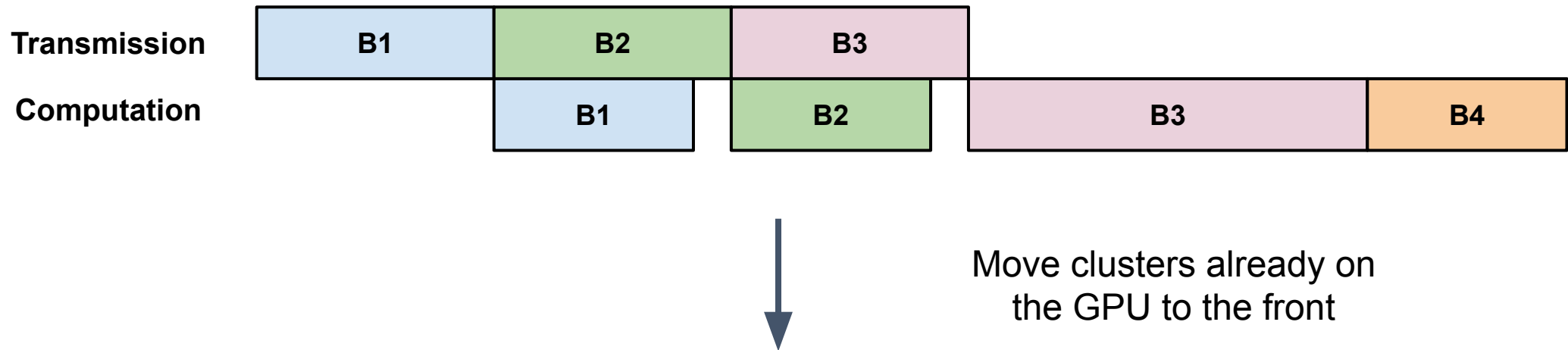
How to Find the Optimal Pipeline?

- Intractable to jointly compute optimal **order** and **grouping**
- Split into two subproblems to approximate:
 - **Identify order** of **clusters** first
 - **Then group ordered clusters** using dynamic programming
- Create profilers (linear models) to predict transmission and execution time of cluster
- Empirically close to optimal

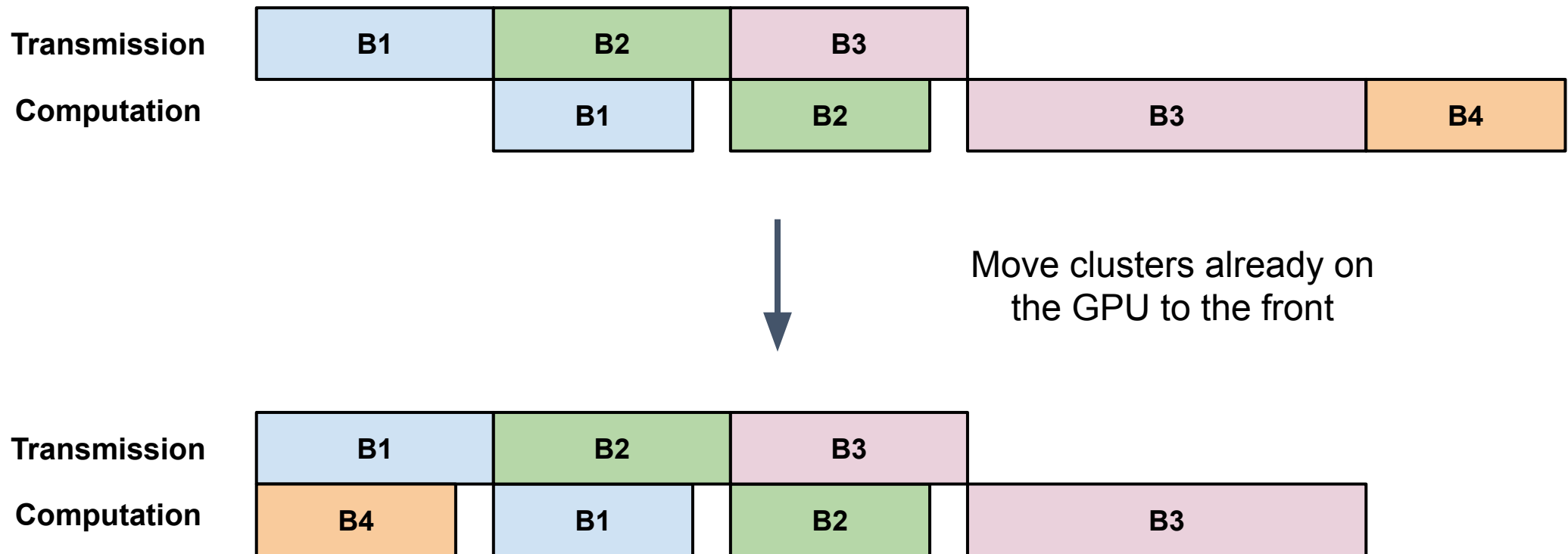
Solution: Identifying Cluster Order



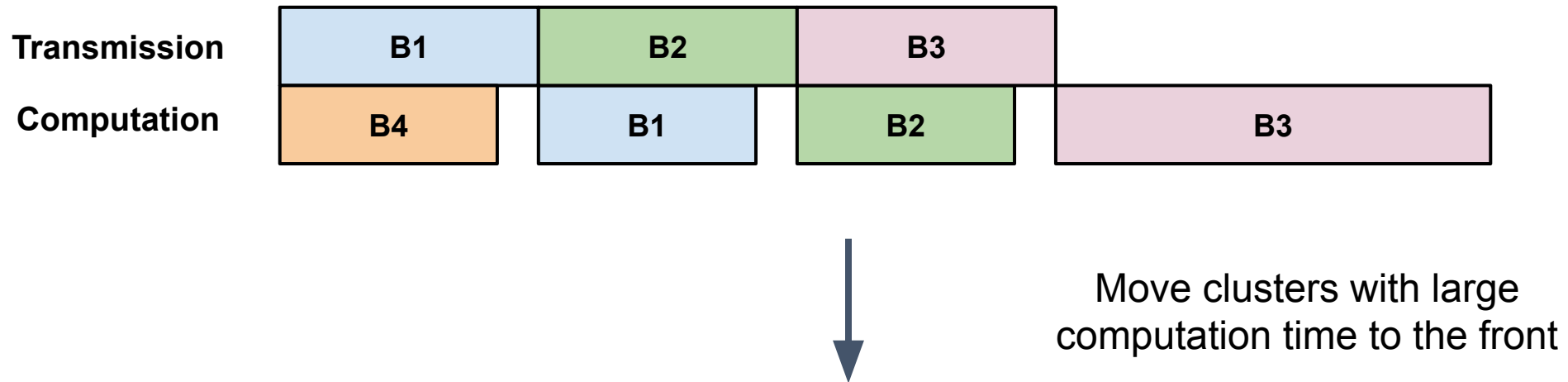
Solution: Identifying Cluster Order



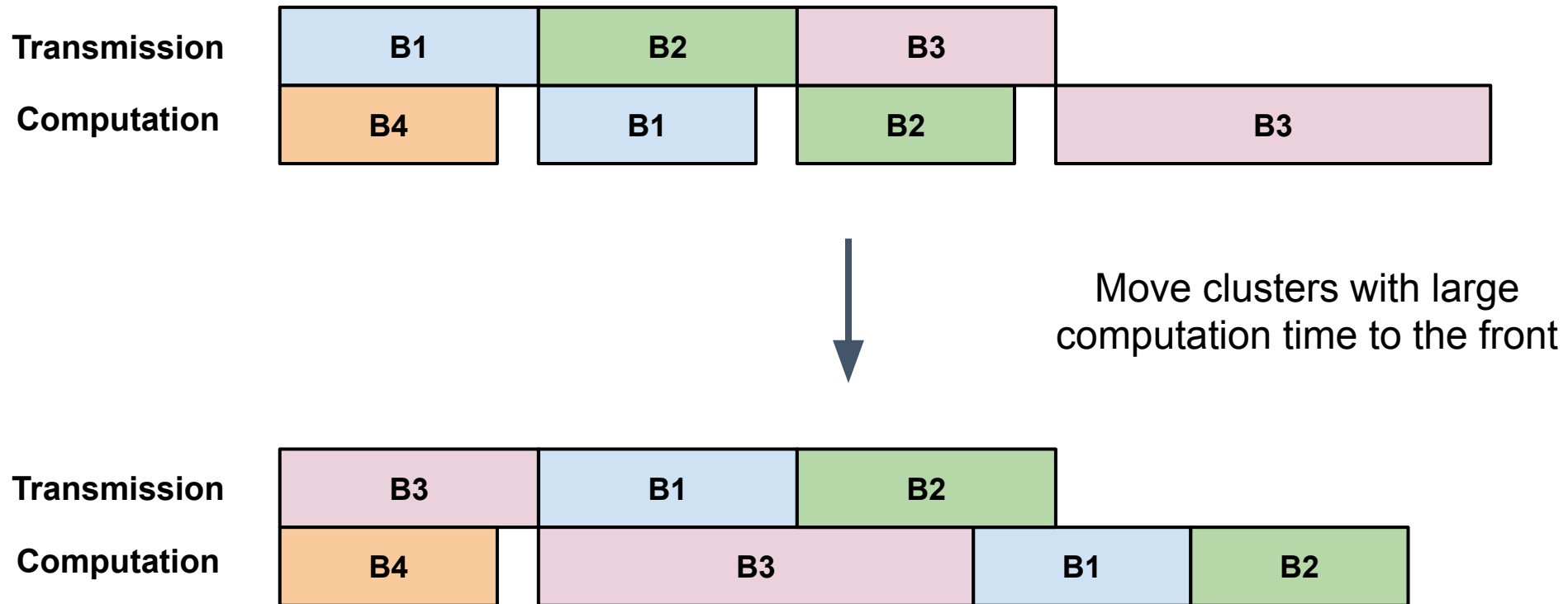
Solution: Identifying Cluster Order



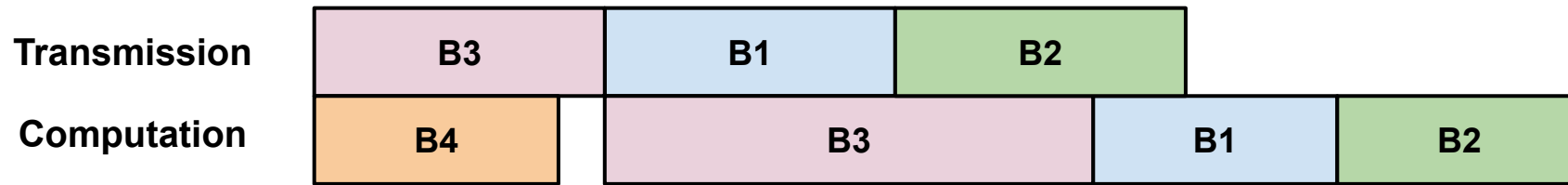
Solution: Identifying Cluster Order



Solution: Identifying Cluster Order

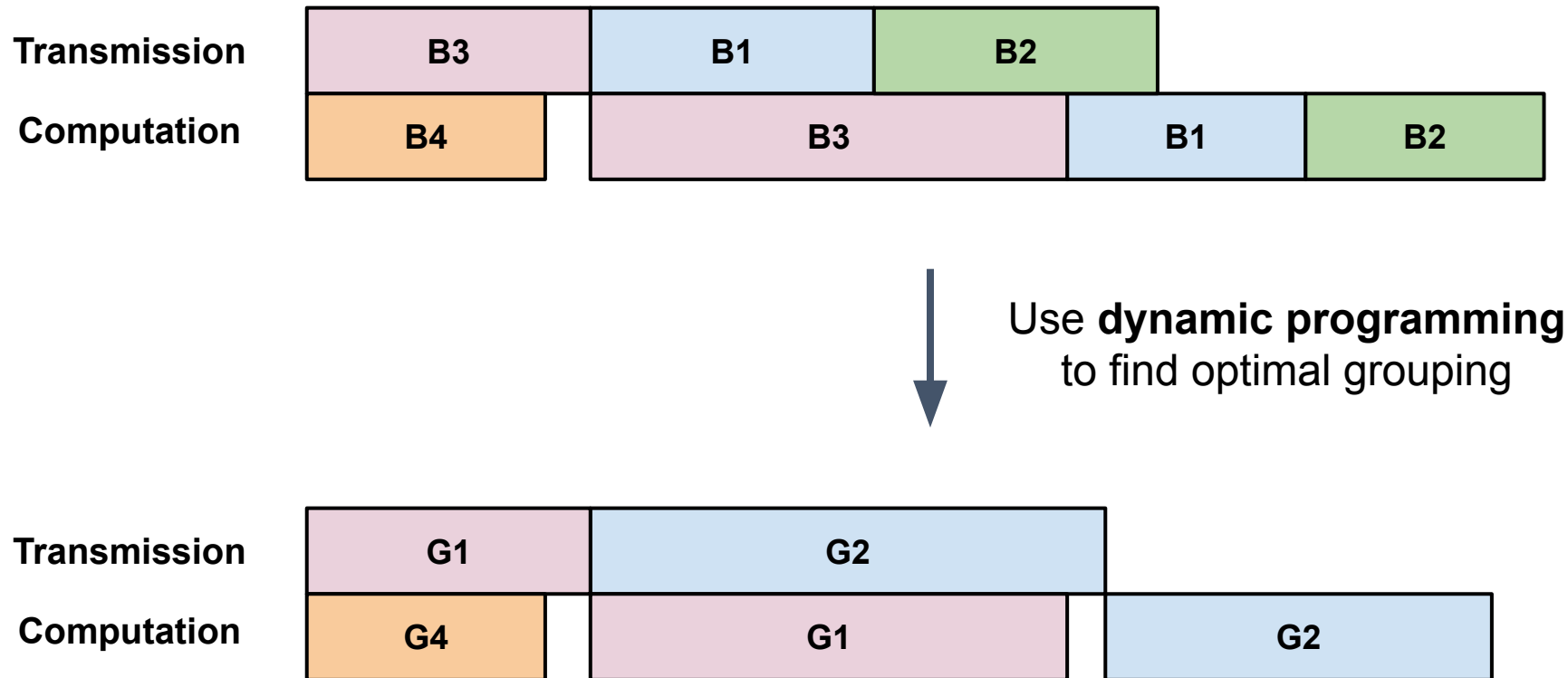


Solution: Identifying Cluster Order



Overheads of frequent invocations and synchronization
between transmission & computation!

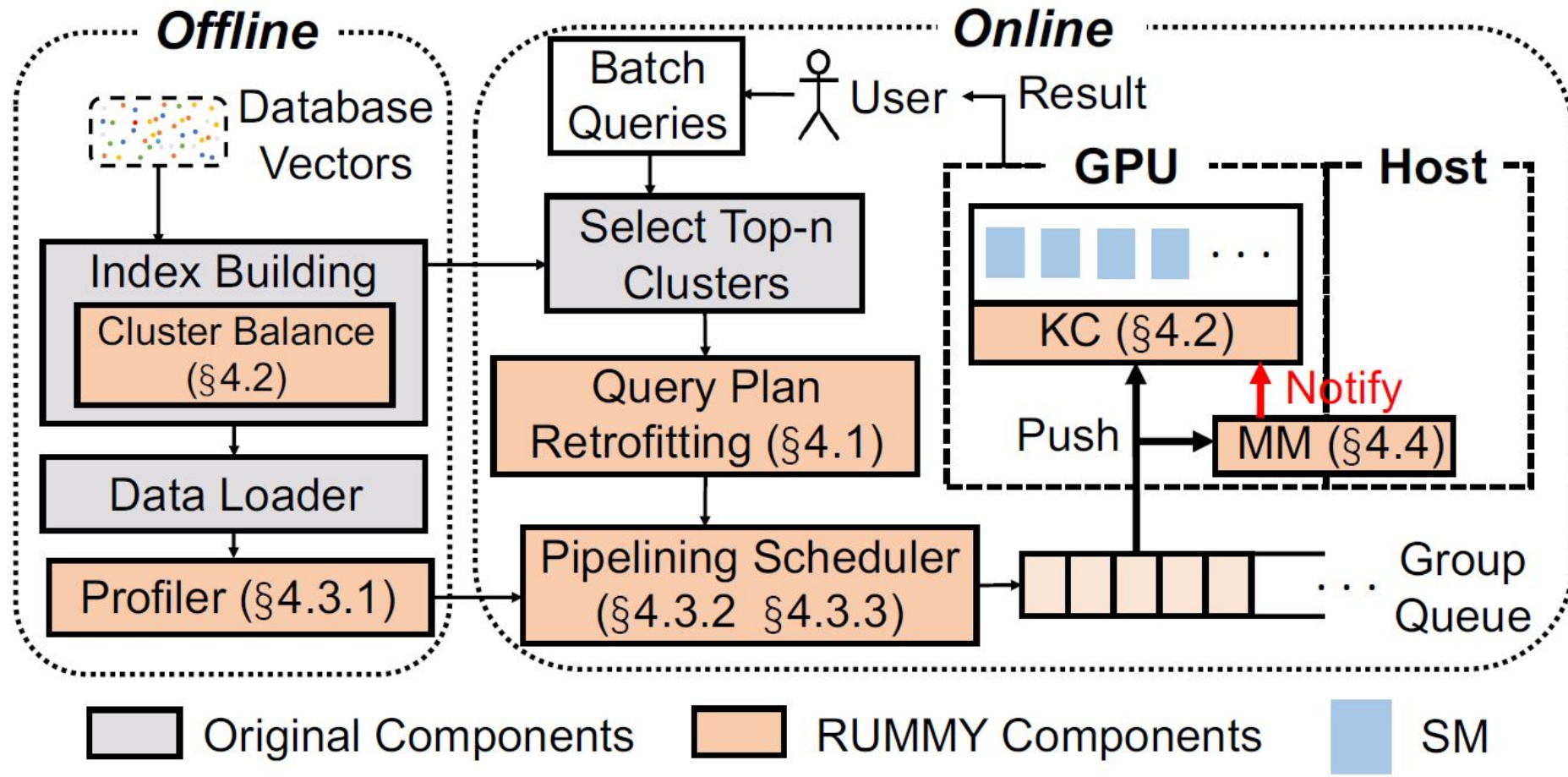
Solution: Identifying Cluster Order



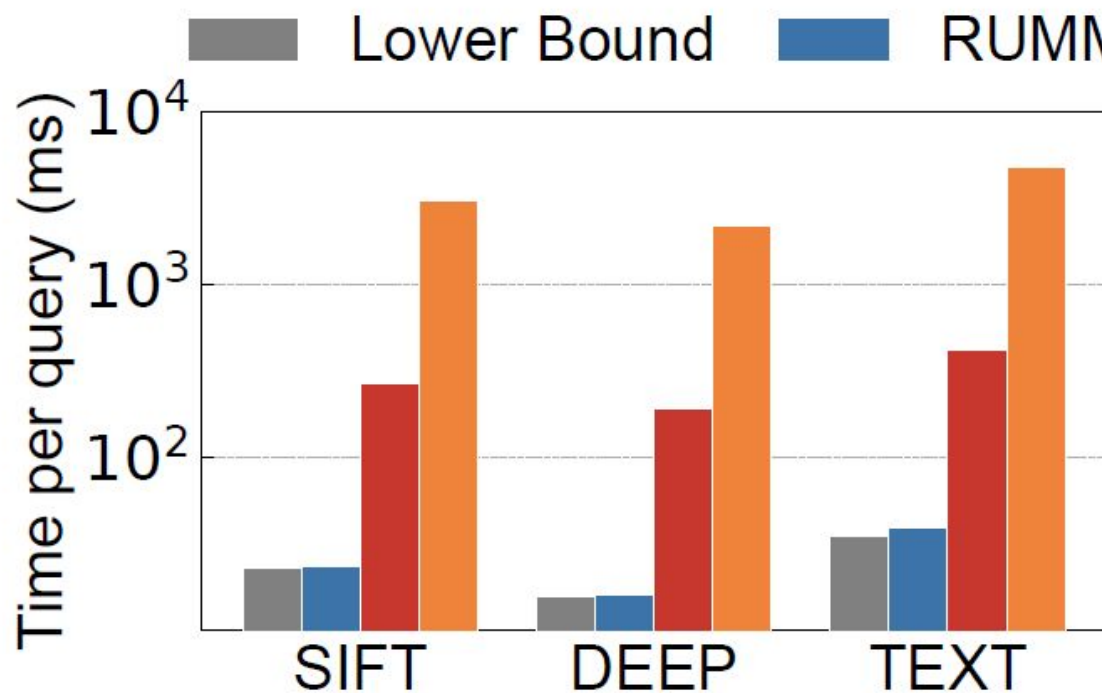
Tailoring Memory Management for Vector Queries

- **Page size** based on **size of balanced cluster**
 - Avoids fragmentation
- GPU page replacement:
 - Within batch, **pin pages required in future**
 - Across batches, **pin *hot* pages**
- **Pin host memory** before transmission to avoid paging to disk

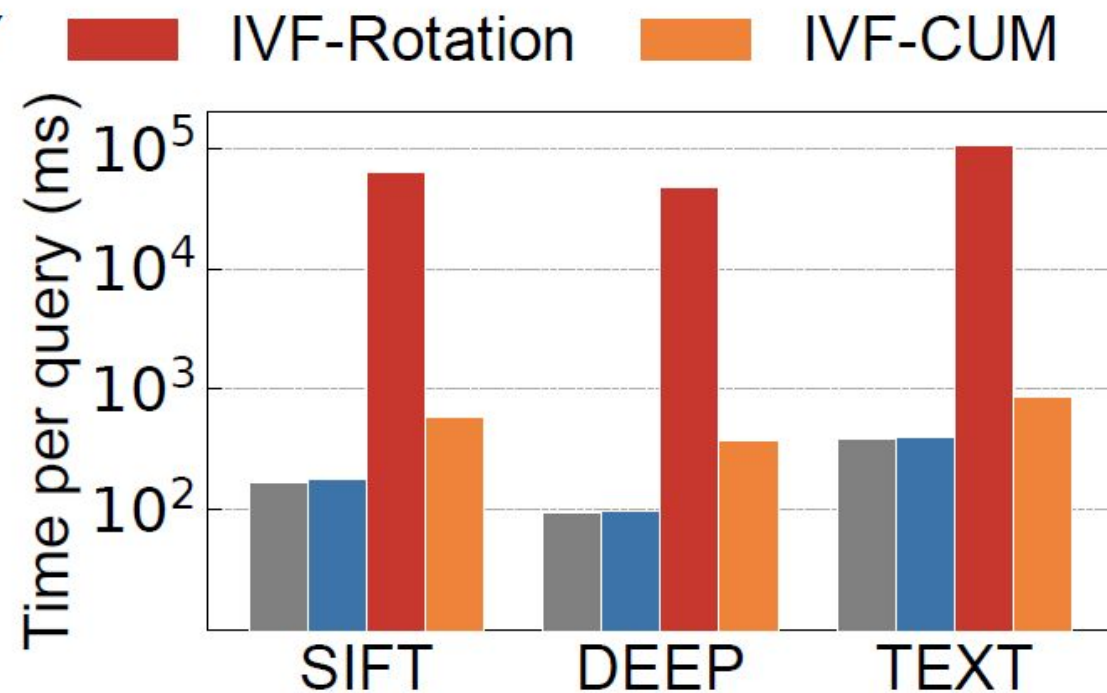
Putting It All Together: RUMMY



Evaluation



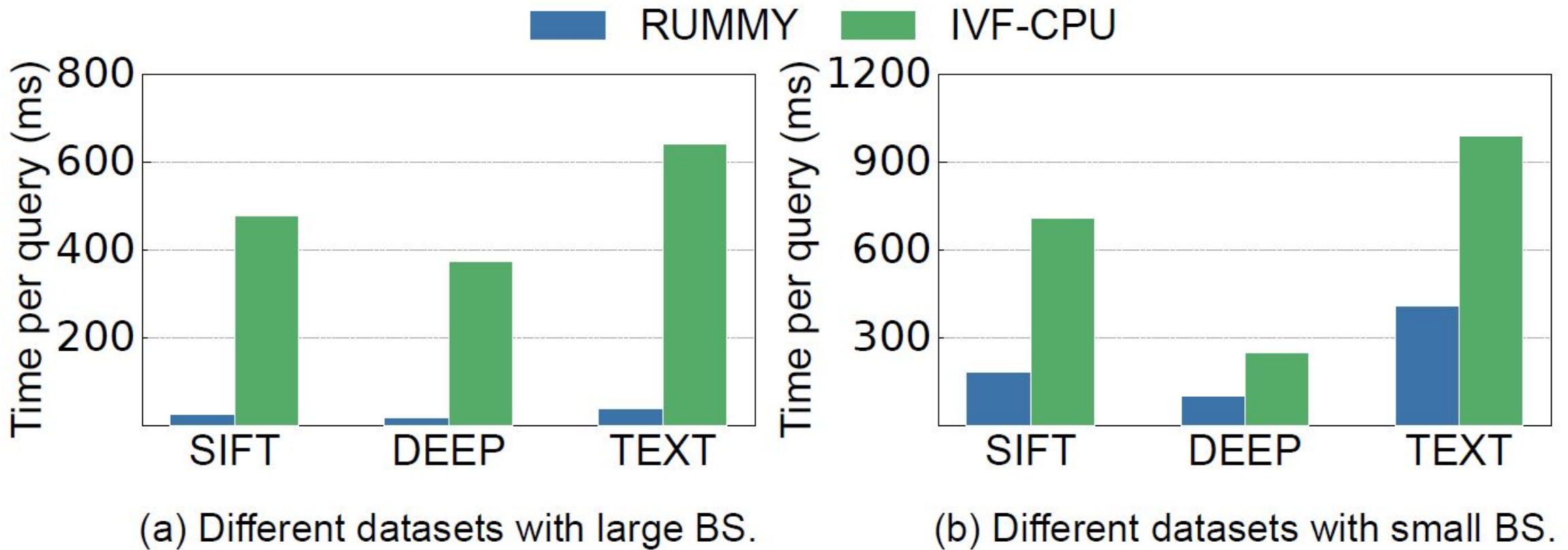
(a) Different datasets with large BS.



(b) Different datasets with small BS.

Overall performance on GPUs

Evaluation



Comparison between RUMMY and IVF-CPU

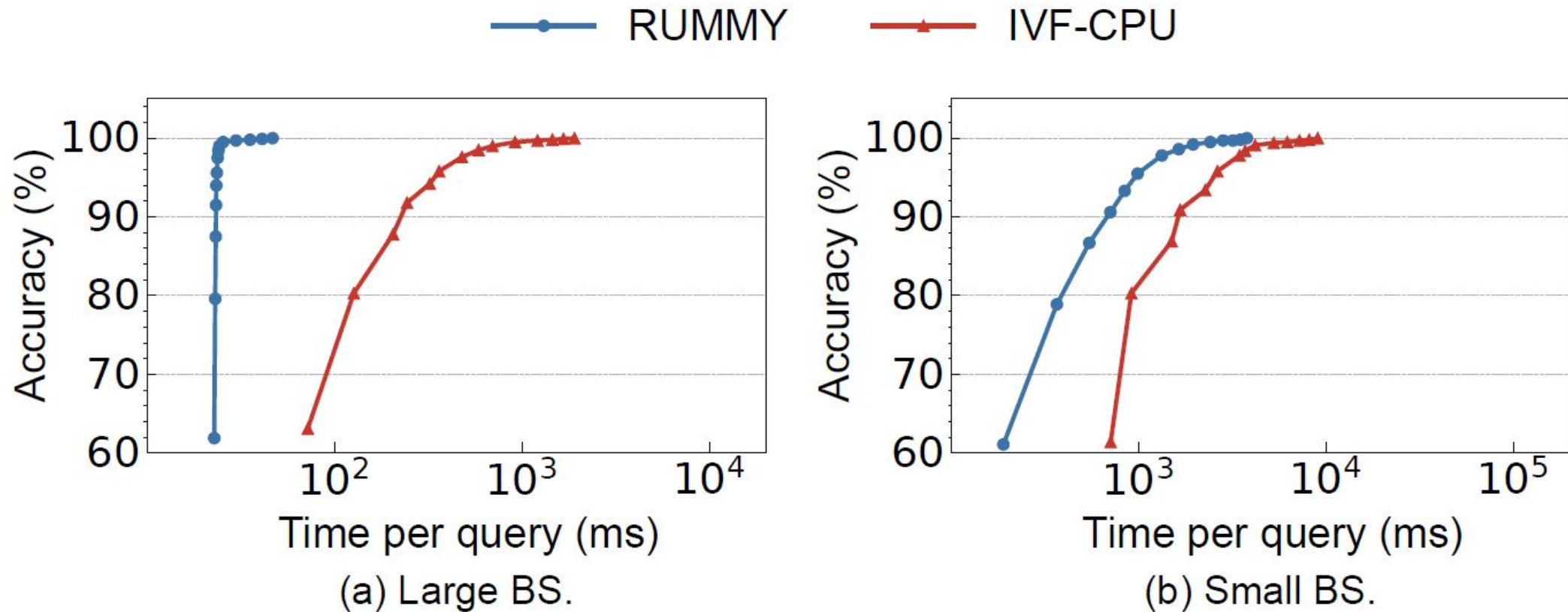
Limitations

- No exploration of multi-GPU configurations
- No discussion of vector quantization methods
- No intelligent scheduling of queries (queries are batched in FIFO, and then retrofitted and pipelined)
- Change in data might require rebalancing of clusters and re-training the profiler
- Poor explanations in places; confusing notation and terminology

Specifically, per-dollar performance is calculated as $\frac{Price}{Time\ per\ Query}$, where *Price* is the AWS EC2 On-Demand hourly rate [54].

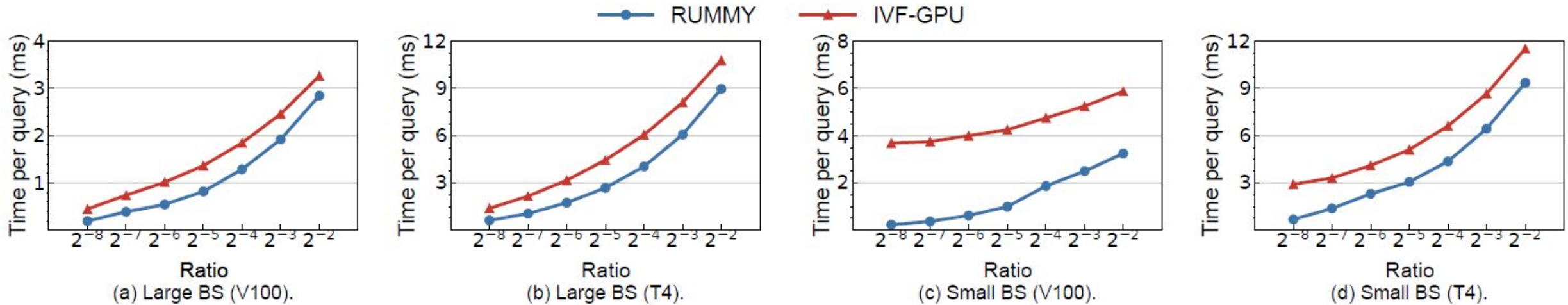
Appendix

Evaluation: Across IVF Configurations



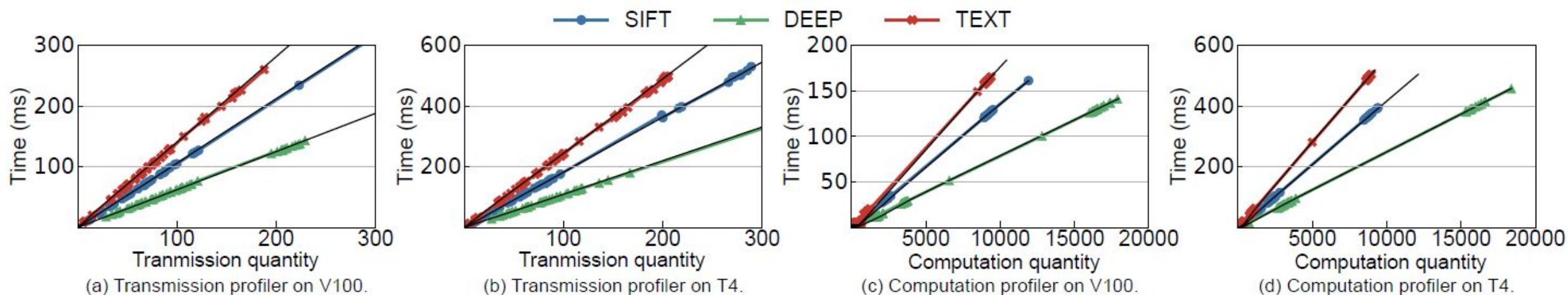
Comparison under different accuracy

Evaluation: Dynamic Kernel Padding & Balancing



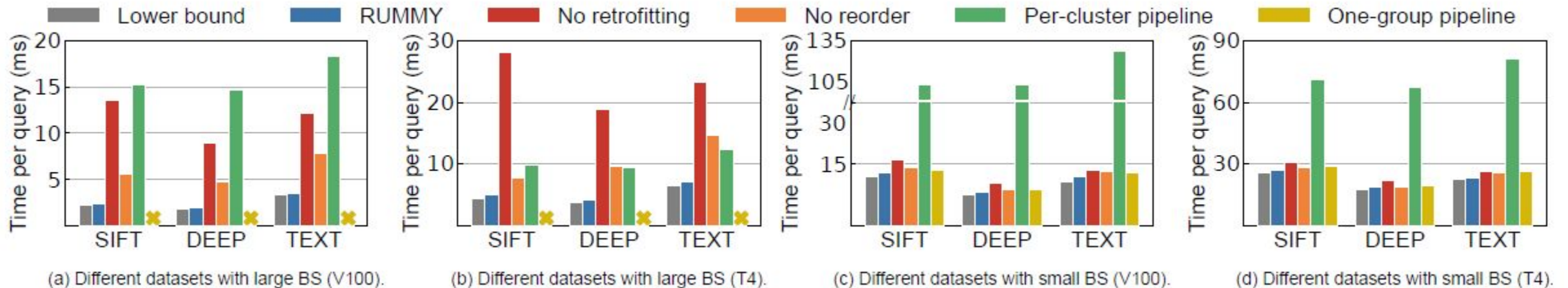
Effectiveness of RUMMY's dynamic kernel padding with cluster balancing

Evaluation: Profiler



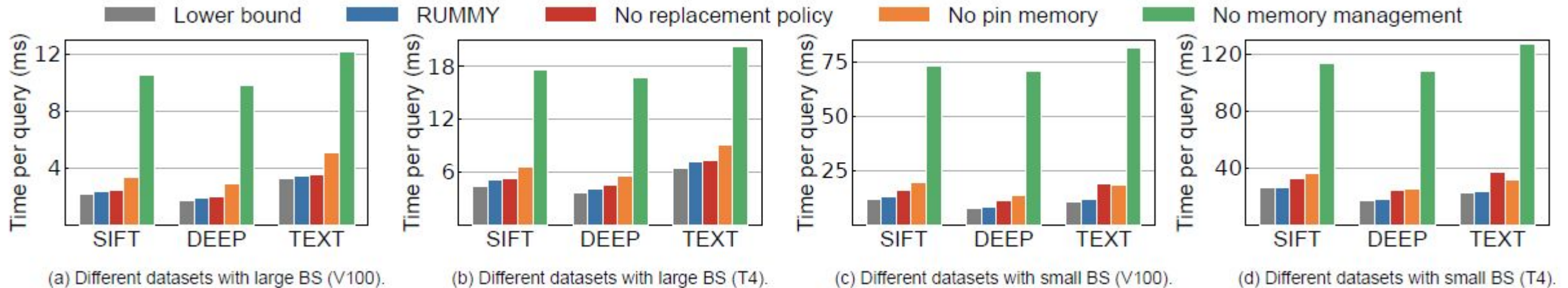
Effectiveness of RUMMY's profiler

Evaluation: Reordering, Grouping, Retrofitting



Effectiveness of RUMMY's reordering and grouping with cluster-based retrofitting

Evaluation: Memory Management



Effectiveness of RUMMY's GPU memory management

ANN on GPU vs CPU

Batch Size	IVF-GPU (T4)	IVF-GPU (T4)	IVF-CPU (r5.2xlarge)	IVF-CPU (r5.2xlarge)	HNSW-CPU (r5.2xlarge)	HNSW-CPU (r5.2xlarge)
	Time per Query (ms)	Per-Dollar Performance	Time per Query (ms)	Per-Dollar Performance	Time per Query (ms)	Per-Dollar Performance
8	0.394	3370.81	20.62	96.2	11.37	174.42
32	0.366	3631.12	18.34	108.163	10.5	188.96
128	0.356	3729.54	17.07	116.23	9.79	202.52
512	0.354	3755.30	16.29	121.77	9.47	209.37
2048	0.355	3740.74	16.25	122.05	9.51	208.55

Comparison between GPU and CPU on different ANN indexes